Daniele Fusi

GVE Technical Documentation

Table of Contents

Introduction 1.1

Model 1.2

- Modeling Autographs: Snapshot 1.2.1
- Textual Model 1.2.2
- Diplomatic Model 1.2.3
- Snapshot Sample 1.2.4
- True Text Snapshot Sample 1.2.5

User Experience 1.3

- snapshot 1.3.1
- editing snapshots 1.3.2
- editing operations 1.3.3
- editing features 1.3.4

Technical Documentation 1.4

- architecture 1.4.1

VEdition

Homepage.

Snapshot

This conceptual documentation refers to the representation of a manuscript text mostly or exclusively written by the author's hand himself. All the terms used here are just a technical commodity for the sake of this documentation.

The autograph is central to GVE because it represents its starting point: for Goethe's epigrams, we effectively start from notebook sheets by the author's hand. There can be several versions of the same epigram in this form, whether they come from a single sheet with annotated variants, or from different sheets, or a mixture of both.

Basic Terminology

Let us first use our case study (GVE) to define some terms we use in the technical documentation with a specific algorithmic meaning.

In the case of GVE, the essential source of epigrams is represented by autograph sheets from handbooks. Whatever its physical support(s), here we call the physical support of an autograph text 📌 **carrier**. In the case of our workbooks, a portion, one, or even more physical sheets referring to each single composition represent a carrier.

The main challenge in modeling the content of these carriers is that they do not just represent a linear text; rather, they represent a sort of 📌 **snapshot** of the creative process which in the author's mind would have led to the intended text. So, the carrier is the material support of the snapshot, which potentially contains many versions of a text.

In fact, graphically the snapshot does not just represent a text, but it includes many corrections, rethinkings, or refactorings of it, in the form of various types of 📌 **annotations** (deletions, insertions, transpositions, side notes, etc.) on top of a text.

The snapshot is thus a sort of picture taken from the author's mind in relation to the text being composed at a given point in time. This does not necessarily imply that this is the end stage of the creative process underlying the text. The process may well have gone further on; but this is not reflected by the snapshot, which just represents a picture which froze an instant of this process at a given time.

The snapshot is thus far from being the representation of an undisputed linear text; it's an even chaotic bunch of text and annotations on some carrier. It is up to the reader to make sense of these annotations, and try to deduce one or more versions of the text from his interpretation.

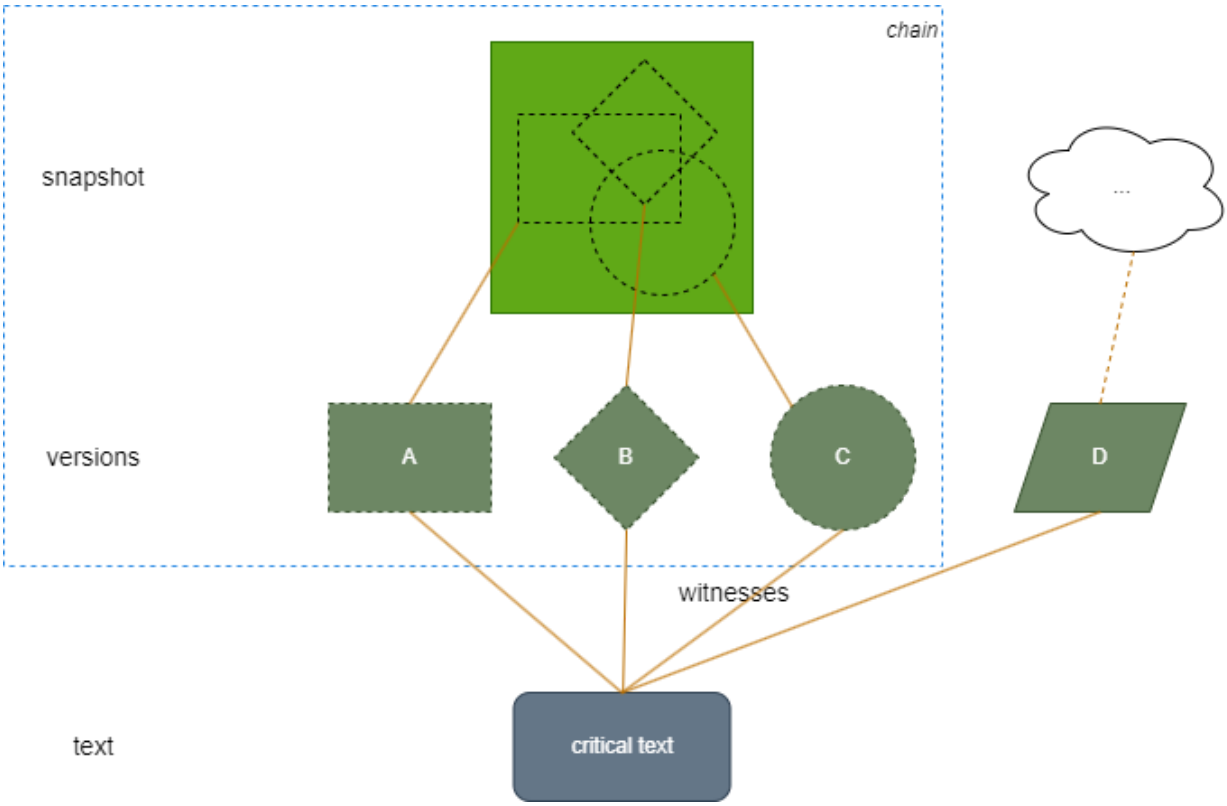
The annotations represent changes on the text; but there often is no way to know their relative order, or even their selection when they represent several competing versions of the text. *It is like having all the ingredients* (our text with annotations), *when we have lost the recipe* (how to select and order the annotations to generate the intended version(s)).

When for whatever reason an authoritative version of the final outcome of all these operations on the text is lacking, in a traditional (paper) edition usually we are forced to forge one, representing the best effort in interpreting our carrier's content. Of course, that would just be a best effort; and it might even be the case that we are not sure in deciding among competing interpretations. In the end, we come up with our best recipe.

In a digital edition instead, we have the power to handle all the rich and complex data preserved by our snapshot, without resorting to compromises. We can represent all the ingredients (text and annotations), all the recipes (annotation selection and ordering), and all their outcomes (resulting versions), while still keeping each as a separate layer, so that everyone can evaluate our text reconstruction, and even provide his own alternatives.

When an authoritative version of the final outcome of all these operations in the text is lacking, in a traditional (paper) edition usually we are forced to forge one, representing the best effort in interpreting our carrier's content.

In a digital edition instead, we have the power to handle all the rich and complex data preserved by our snapshot, without resorting to compromises. We can represent all the ingredients (text with annotations), all the recipes (annotation selection and ordering), and all their outcomes (resulting versions), while keeping each as a separate layer, so that everyone can evaluate our text reconstruction, and even provide his own alternatives. This scenario is depicted in this diagram:



At the top of the diagram we find our carrier, represented by the green rectangle. Here, any part of, or one or more sheets, are the carrier of a snapshot, which potentially defines many versions of the text, here represented by the various shapes inside the rectangle.

As remarked, these versions are not explicitly and clearly defined; they are the product of our interpretation of the snapshot. In our diagram we have extracted 3 versions: A, B, C. At the same time, other versions of what we regard as the same text may come from other sources, whatever their type (other snapshots, other witnesses, previously published versions, etc.). This is the case for version D in the diagram.

So, at the bottom level we find the carrier with its snapshot. On an upper level instead we find the various versions of the text we want to critically publish. These versions might independently exist, or be deduced from one or more snapshots; but for the sake of this architecture that's just a detail. In the end, in relation with the critical text we might want to reconstruct, all these versions work as *witnesses*, just like in a traditional critical edition we deal with different manuscripts and other sources.

So, on a third layer we find the *critical text*, which by definition is a reconstructive hypothesis. This is based on a selection of *versions* representing our witnesses, and located in a lower layer (versions A-D). Then, in the case of autographs we have yet another lower layer (the carrier with its snapshot), which generates some or all of these versions.

> The critical text can be reconstructed at different levels: we can limit our reconstruction to what is attested by our snapshot's versions, when focusing on the text (in our scenario, an epigram) from each single snapshot; or collect versions coming from several snapshots (and optionally from any other source), when focusing on the epigram derived from the critical comparison of the many epigrams collected from our various sources, if this is feasible and required (e.g. to print a single text). This is just a matter of branches selection in the lineage of our text: when focusing on each single snapshot, we may want to publish a reconstructed text (epigram) from its versions, in this single-ancestor lineage; when focusing on many snapshots or on a mixture of other sources, we may want to publish a reconstructed text from all of them, or rather just provide a set of reconstructed texts. This is what happens in GVE, where there is no authoratitive publication of the text where all the versions would converge. So, in this case we will end up with a collection of multiple versions we might name a “synopsis”; and such will be the level above our snapshot for this peculiar edition. At any rate, at the conceptual level of this model this belongs to the layer of reconstruction, based on the layer of witnesses, whatever its source(s).

Thus, the scope of our autograph model encompasses the area defined by the bounding dashed rectangle in the diagram: a carrier with a snapshot, which potentially defines many versions of a text. What we want from our model is a representation of that snapshot, both on the textual and on the graphical side.

Once this model satisfies all our requirements, it will be the foundation for structured and layered representation of its different parts and planes (textual and diplomatic, objective and subjective) at a high granularity level (up to the single character or visual trait). From this, we will then be able to generate many outputs including TEI documents with various schemas, complex and interactive UIs, or any other type of export.

- textual model
- diplomatic model

Textual Model

Model Requirements

Ideally, the main requirements of this model are:

1. we want to **represent the snapshot** itself in a *computable*, generative way, defining the process which builds the reconstructed output.
2. in this process, we want to **preserve** as distinct, structured data our *ingredients* (text and annotations), *recipes* (annotation selection and ordering) and *outcomes* (text versions).
3. we want to represent the snapshot annotations both in their **textual and visual form**. This is because the graphical representation of each annotation is semantic information, which should be encoded next to the textual variant it represents. By preserving both these layers of data we provide a richer model, which can be inspected by scholars used our digital edition as a tool, or just leveraged by more casual users as another way of looking at the text and its creation process.

A good model should meet all these requirements using the simplest possible structure, which implies *raising the abstraction level* so that even the finest details can find a generalized and consistent representation.

This also allows our model to lay the foundation of a paradigmatic approach, leading from a single case study to a reusable tool.

Core Chain Model

Our model core is represented by a general purpose data structure we call 🪢 **chain**. The task of this data structure is *representing multiple linear combinations of a set of entities*. We represent these entities as **nodes** of a **graph**, connecting them into linear sequences.

In our specific case, our entities are characters, and their linear combinations build a sequence representing a text. So, a chain has the power of representing multiple texts (i.e. multiple versions of what we regard as the "same" text in the course of its composition) at the same time; and this is right what a snapshot on a carrier does.

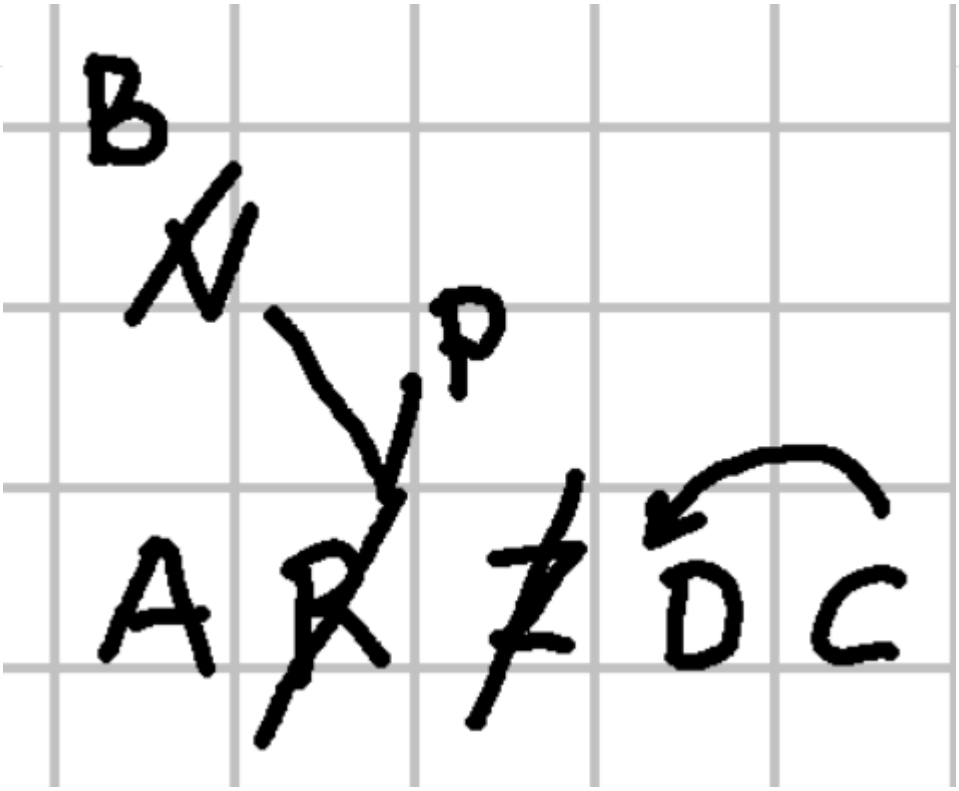
In technical terms, we could describe this structure as a *tagged multigraph linked list*:

- it's a *linked list* because each node has at most one child in each version of the list.
- it's a *multigraph* because it allows multiple edges (links) between nodes.
- it's *tagged* because each version of the list (each set of links) has a unique tag.

Operations

Let us see how this works by using a simplified diagram. Consider the image below, which I created by first writing characters `ARZDC`, and then making some annotations on it, representing variations of the text.

> A text being a linear sequence of characters, we don't care creating a true text in a specific language here; we just deal with such sequences, so that our examples are shorter and more to the point.



This snapshot provides annotations which can be interpreted as operations changing the **base text**, which is just what happens to be the starting point of our digital representation. So, "base text" here has no genetic or philological implication; it is just a practical device in our model.

As our model represents multiple versions of what we consider the "same" text, this means that it represents a set of changes in it, from the first to the last version in the author's mind. Again, "first" and "last" here don't imply that these are the two edges of the line representing the whole creation process; rather, they are the two edges of the segment in this line which is covered by our snapshot. When representing multiple versions of a linear sequence in terms of changes of a starting sequence, of course we need right that: *a sequence to start with*. This is our "base text".

Often, this base text will correspond to what in a relatively tidy snapshot looks like a line in the main flow of text, used as the anchor for annotations. In our diagram, this is the line where I wrote `ARZDC`. This does not necessarily imply that I *first* wrote `ARZDC`, and *then* all the rest; I might well have written `AR`, then some of the annotations above it, and then the rest of the letters in the same line (`ZDC`); or I might have followed any of the other order of actions I could ever imagine for writing the text photographed by our snapshot. This is not relevant here; all what we want is a sequence to start with, and on the ground of both visual and textual representations (and optionally their meaning, when dealing with a real text) I happen to pick `ARZDC`.

Having defined our base text, let us now focus on the **annotations**. The annotations on the sample snapshot clearly represent *operations* on the text, which in a visual way lead to generate different versions of it:

- the slash on `Z` represents a *deletion operation*.
- the slash on `R` is part of a *replacement operation*. Of course, in simpler terms a replacement is just a composite operation, made of a deletion followed by an insertion. Yet, from our more synthetic point of view we prefer to look at it as a replacement. In fact, that's what our snapshot visuals aim to represent; we have a slash on `R`, and two segments originating from it and leading to two alternative letters: `V` on the left, and `P` on the right.
- in turn, the slash on `V` is another *replacement operation*: here it gets replaced by `B`. Note that in this case we are nesting an operation on top of another operation; which, in turn, happens to be on a branch representing two alternative operations. One branch stems from `R` and goes across `V` up to `E`; the other one stems from

`R` and goes up to `P`. It is our conventional interpretation of these visuals which defines these operations, and in some cases even hints at their relative order.

- finally, the arrow on top of `DC` represents a *movement operation*: `C` is moved before `D`. Again, this is a higher-order, complex operation that we could just represent as a deletion (delete `C`) and an insertion (insert `C` before D); but we prefer a complex operation here, as it best fits what we clearly understand about the writer's intention when seeing the arrow "moving" `C` before `D`.

So, all our *annotations can be modeled in terms of operations*, which affect our chain.

Also, we do not just limit ourselves to basic operations like insertion or deletion. Even if these would be all what is strictly required to represent any change on a linear sequence, we want a model as near as possible to a human-friendly representation and interpretation of our snapshot.

This is because operations here essentially belong to the layer of interpretation, and interpretation here is a human fact. Here, we are looking at our *snapshot*, and inferring operations from its visuals.

The *snapshot is objective, empirical evidence*: we effectively take a photographical image of it, and we could even geometrically represent all its visuals.

Conversely, *operations are our interpretation of these visuals*, which define the ingredients of our lost recipe: the letters of our text, and the operations which change their combinations into sequences.

We thus want such operations to be as close as possible to a human-friendly way of thinking; and for this reason we allow for many complex, higher-order operations, as far as they best fit our needs.

Chain

So far, we have modeled our snapshot and its *ingredients*: we have letters, and operations on them. Some of these operations just move letters around; others can delete them, or add new ones.

Our next challenge is representing a *recipe*. In our sample snapshot, our ingredients are:

- the base text `ARZDC`;
- the deletion of `Z`;
- the replacement `R=V`;
- the replacement `V=B`;
- the replacement `R=P`;
- the movement of `C` before `D`.

As we have seen, these operations already imply a degree of interpretation; but in most cases this is a trivial interpretation, which poses no issues.

The main issue for interpretation is instead *selecting and ordering our ingredients*, that is, building a **recipe** for handling them in a way leading to a final outcome, which in our case is a version of our base text.

In other terms, we have six ingredients listed. Among them, five are operations. Now, should I select all of them to build a specific version, or just a subset? And what would be their order of application? We need at least one recipe, or even more than one; each of them will represent an interpretation of our snapshot leading to a specific staged **version** of our text.

So, recipes are completely up to our interpretation: of course, they are constrained by the number and type of available ingredients; but within these limits, any combination is possible. It is up to the scholar's judgement to consider all the complex factors behind a recipe, and define it, thus generating one or more versions of a text.

Notice that our model will always include all the ingredients, as we want to fully represent our data at the snapshot level. We do not want any of our interpretations to have destructive effects on our data: just like the snapshot presents all the ingredients mixed together, we want to collect and preserve all of them as distinct entities. This will

not only ensure that our model fully represents our data; but also that we can add as many interpretations as we want on top of them, or even allow others use the same data to come out with their own interpretations.

So, the challenge here is having a model which allows scholars to define a recipe for combining ingredients to get any number of distinct text versions, while still containing all the ingredients in the same box. In a sense, if you think of a text as a string (a linear sequence of) characters, it is like getting a *time-enabled string*, i.e. a single data structure with the ability of representing not just a single sequence, but many different ones, all within the same box. Now, this is right what the *chain* data structure is for.

This data structure consists of a *linear sequence* of **nodes**, each connected to the previous and the next node by a **link**. In our case, nodes are characters.

So, a chain has a set of nodes, and a set of links connecting them. Notice that none of these sets is ordered; it is the link which defines the order of a node in a sequence.

To allow for multiple sequences in the same set, each link has a *version tag*, which adds the dimension of time to this structure. So, essentially any change on the texts represented by the chain is made by adding new layers of links, where each layer is just defined by the link's tag.

For instance, let us start from our base text, `ARZDC` : if we represent nodes inside square brackets, and links with the two characters they connect separated by a colon, the base text (let's call it `v0`) is:

```
-:A [A] A:R [R] R:Z [Z] Z:D [D] D:C [C] C:-
```

This means that our chain contains 5 nodes (`A` , `R` , `Z` , `D` , `C`) and 6 links:

- `-:A` , i.e. the link connecting the start edge of the chain to the first node;
- `A:R` , link between `A` and `R` ;
- `R:Z` , link between `R` and `Z` ;
- `Z:D` , link between `Z` and `D` ;
- `D:C` , link between `D` and `C` ;
- `C:-` , i.e. the link connecting the end edge of the chain to the last node.

All these links are tagged with version `v0` . Once we have filled the chain's sets, we can generate any of the text versions by just following all the links tagged for that version, starting from the first link and moving from node to node.

For instance, to get the text for version `v0` we start from link `-:A` , which leads us to node `A` ; then, we get to link `A:R` , which leads us to node `R` ; and so forth, until the end link (`C:-`). As we move from node to node, we collect characters; in the end, the collected characters represent our sequence, `ARZDC` .

If now we want to represent another version of the text, as resulting from the deletion of `Z` , all what we have to do is adding a new set of links for the next version (let's call it `v1`):

- `-:A` , i.e. the link connecting the start edge of the chain to the first node;
- `A:R` , link between `A` and `R` ;
- `R:D` , link between `R` and `D` . This bypasses `Z` , with the effect of "deleting" it from the text;
- `D:C` , link between `D` and `C` ;
- `C:-` , i.e. the link connecting the end edge of the chain to the last node.

The nodes set is unchanged; `Z` itself is still there. Yet, we have bypassed `Z` in the sequence by short-circuiting the link between `R` and `D` . The result is that when building our version text, we no more traverse the `Z` node, so that we get `ARDC` .

Other operation types, like additions or replacements, also imply the addition of nodes; but for the rest they work the same, by adding new links in the set. For instance, if we are going to replace `R` with `V` , we will:

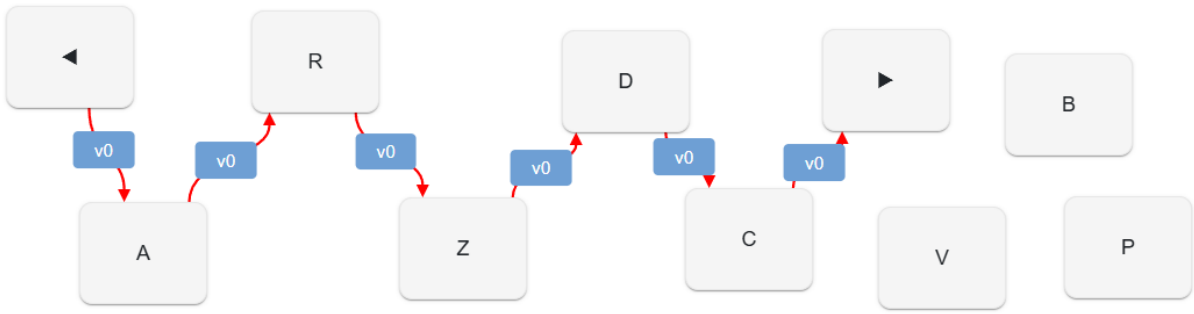
- add a new node, `V`.
- add a new set of links for a new version (tagged `v2`):
  - `-:A`
  - `A:V`
  - `V:D`
  - `D:C`
  - `C:-`

Again, we just added items to our chain's sets, but the result is that we have defined another sequence for it, corresponding to yet another text version: `AVDC`.

A demo UI for this data structure can let you play with operations on it, showing among other data also a diagram representing nodes and links in the chain. So, let us consider the full set of operations in our sample snapshot, and build the corresponding diagrams. Our base text is `ARZDC`, representing our version 0 (`v0`); operations are:

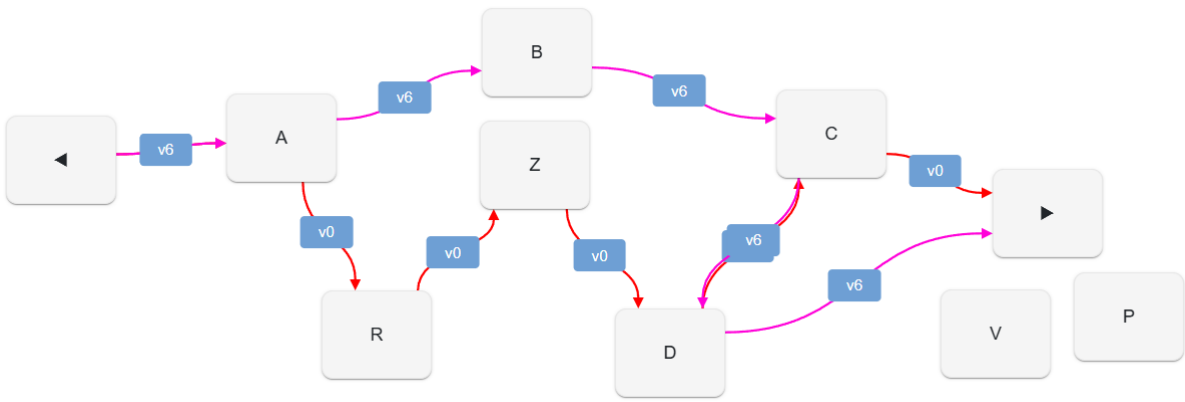
- the deletion of `Z`;
- the replacement `R=V`;
- the replacement `V=B`;
- the replacement `R=P`;
- the movement of `C` before `D`.

The diagram below shows the representation of version `v0` via nodes and links once all the operations have been executed, in the order specified above:



For clarity purposes, the diagram also shows start and end edges as nodes (with left and right triangles); yet, in the set there are no such nodes, but only nodes for letters `A`, `R`, `Z`, `D`, `C`, `V`, `B`, `P`. Note that some of the nodes are left out from the sequence; they still belong to the nodes set, but they happen not to be used for building the text of `v0`. This is because they were effectively added later, by replacements. So, following links you get `v0` text `ARZDC`.

Let us now consider the last version produced after all the operations have been executed; this is version 6 (`v6`). To avoid overcrowding the diagram, here we have filtered out links not belonging to `v0` or `v6`:



> Note that when showing more than one versions some links can be overlapped in the diagram, according to how you move the shapes around. The UI diagram is interactive, so you are free to drag shapes wherever you want, maybe revealing or hiding some of them. In this case I have moved nodes to make easier to read the different versions; yet, some links are overlapped: this is the case of `-:A` for both `v0` and `v6` : `v6` overlaps it completely, but it's still there (just compare the previous diagram showing just `v0`).

Now you should start to see the chain's ability to represent multiple versions at the same time. Here we have two sub-sets of links: those tagged with `v0` (red), and those tagged with `v6` (magenta).

If we follow `v1` links, we traverse these nodes:

1. `A`
2. `R`
3. `Z`
4. `D`
5. `C`

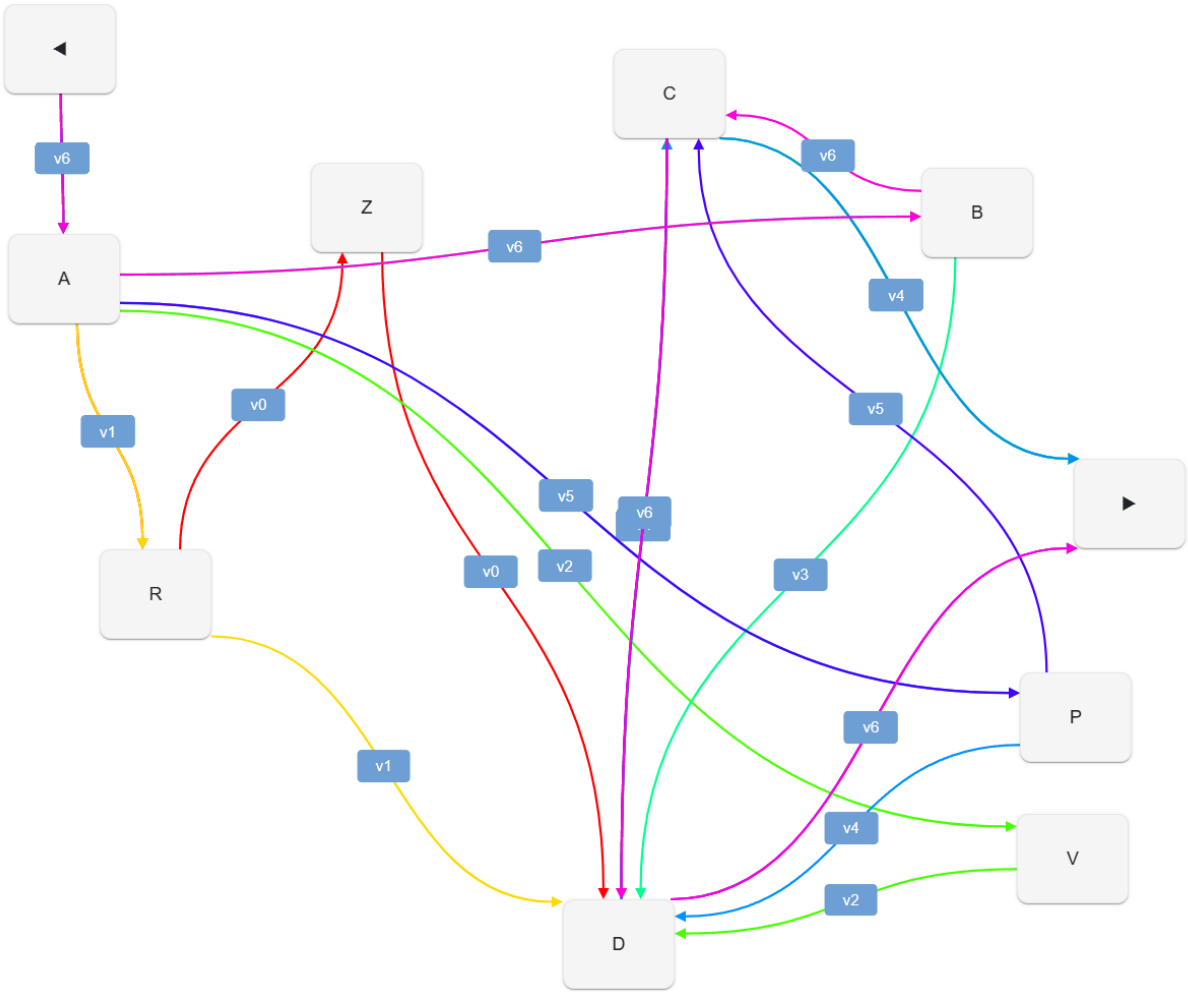
If instead we follow `v6` links, we traverse these nodes:

1. `A`
2. `B`
3. `C`
4. `D`

While the nodes set stays the same, our *links define the recipe: they tell which nodes to pick from the set, and in which order*.

Thus, we have gained the ability of representing multiple strings with this sort of time-enabled strings: in a sense, the chain is a multi-versioned string.

If we remove filters from the diagram, and look at all the versions from `v0` to `v6` at once, here is the result:



The links now define 7 different versions of our text:

1. v0: `ARZDC`
2. v1: `ARDC`
3. v2: `AVDC`
4. v3: `ABDC`
5. v4: `APDC`
6. v5: `APCD`
7. v6: `ABCD`

So, a single chain structure represents 7 variants of the same text; these are 7 recipes, all using the same ingredients, which are always preserved in the box, whatever recipe we decide to follow.

Whenever we apply an operation to our chain, we get a new version out; so we have the fully detailed text generation process, step by step, as defined by our interpretation. Of course, *not all the versions output by this detailed process are eligible for representing what we really want to define as a text "version"*; maybe in our interpretation we have isolated 3 versions, corresponding to `v2` (`AVDC`), `v5` (`APCD`), and `v6` (`ABCD`), while discarding the other as just generative steps between them. The chain has a mechanism for this selection, but we must first introduce another feature of them before delving into its details.

To sum up, we have thus met the first two requirements of our model:

(1) represent the *snapshot* itself in a *computable* model.

(2) preserve as distinct, structured data our *ingredients* (text and annotations), *recipes* (annotation selection and ordering) and *outcomes* (text versions).

Yet, we still have to address the last one:

(3) represent the snapshot annotations both in their *textual* and *visual* form.

This implies delving into more details about our chain's operations, which will also show more about the data they carry and inject in their output.

Metadata

As we have seen, the chain structure is a container of any number of versions of a text, and operations target it by adding new links and optionally new nodes to its set. So, chain operations are abstractions on top of the chain structure, used to represent higher level editing operations on the chain's text.

As already noticed, the operations being the first act of interpretation of our snapshot we do not limit ourselves to the basic ones only (insertion and deletion); rather, we prefer a full set of higher order operations, which best fit the nuances of our descriptive model.

So, there are 8 **types** of operations (listed with their corresponding operator in their text-based representation):

- replace (`=`): e.g. `1x2=X` .
- delete (`-`): e.g. `1x2-` .
- add before (`+[`): e.g. `1x1+[X` .
- add after (`+]`): e.g. `1x1+]X` .
- move before (`>[`): e.g. `1x2>[5` .
- move after (`>]`): e.g. `1x2>]5` .
- swap (`<>`): e.g. `1x2<>3x3` .
- annotate (`:`): this is a do-nothing operation which affects only metadata by updating features; e.g. `3x2:[note=sample]` .

Given that each operation affects the chain producing a different version of the text, we might also need to attach specific **metadata** to either that version as a whole, or to the nodes affected by it.

So, operations can also carry metadata to be attached to their output; these might refer to any kind of data useful to be carried on the affected nodes, and/or on the chain context as a whole. For instance, a single operation might be attributed to another person (e.g. a suggestion from Schiller about some change in an epigram), or it might be written using a different ink, or receive some additional editorial note about its interpretation, etc.

All the operations thus share a common set of metadata, represented by:

- generic **features**. This is an open-ended set of any type of features, modeled as generic name/value pairs. So, for instance the ink color of an annotation represented by an operation might be represented as a name of `color` , and a value of `red` .
- **source** metadata: these metadata are specialized to represent the source for the variant implied by a specific operation. The model of a source item is similar to that of a generic apparatus, and contains:
  - *ID*: an identifier for the source. For instance, it might be the identifier used for Schiller, or for some other witness.
  - *type*: the type of source (a person, a witness, etc.).
  - *rank*: the probability rank attributed to the identification of this source.
  - *note*: an optional free text note.
- **diplomatic** metadata: these metadata are specialized to represent the *visual* aspects of an operation on the snapshot's carrier (see the diplomatic portion of the model).

Features

The feature carried by an operation (and other objects in the snapshot model) is a bit more complex than just a name/value pair. Among its properties, a feature has a specific **set policy**, which drives the behavior of the operation when updating features in their sets (=the context set, when they are global; or the nodes set, when they target the affected nodes).

The set policy can have any of these values:

- **single**: when a feature is added to a set, and a feature with the same name already exists in it, the existing feature gets replaced with the new one. For instance, the certainty rank is a single-policy feature, because logically you can just have a single level of certainty; if you set a new level, you are overwriting the value of the previous one. Also, the version feature is used to mark a specific output as a staged version; so, whenever a new version is defined, the old version value is replaced by it.
- **multiple**: when added to a set, the feature is just added to the set, whether another one exists with the same name or not. For instance, a `log` feature is used in our examples when we want to trace the operations effects on the generated texts. In this case, each operations logs a new entry and we want to preserve all the entries as they accumulate. So, this is a multiple feature, and whenever a new log entry is added all the existing entries are preserved.
- **single first**: when added to a set in a batch, and a feature with the same name already exists, the first one being added replaces all the existing ones, while the following ones to be added just get added as multiple. This has meaning only when adding several features at once. Otherwise, it just behaves as single.

> Some features get automatically projected onto their targets when applying an operation. This is the case of probability rank and source features. Source features also provide an example of the single-first policy: as every operation can have multiple sources, we want to replace the sources from previous operations the first time we add a source from the current operation, but then add all the other sources included by that operation. Should we use a single set policy, we would end up preserving only the last-added source, which is not what we want.

Operations DSL

Even though GVE provides a UI to edit such operations, it also allows a text-based representation for them using a minimalist domain specific language (DSL). This text format is used to represent their basic metadata, including features, but excluding specialized metadata (source and diplomatic). It is mostly used to speed up editing, or for testing or diagnostic purposes in code.

The general text-based syntax for these operations includes these components, from left to right:

(1) `(ITAG:OTAG)` : these are the optional input and/or output **version tags**. They represent the input and the output versions in the chain before and after applying the operation. You can specify none, just one, or both. Usually there is no need to specify version tags, as the system can generate tags, and automatically increase their number (`v0` , `v1` , `v2` ...) after each operation is applied. You might want to specify a version when you want to use a special name for it (which anyway is discouraged, as it would introduce variants in the expected naming scheme for tags), or when you want to start from a version which does not correspond to the latest generated version.

(2) `ATxRUN` or `@ATxRUN` (where `AT` is required): the **range** of characters to select as the input of the operation. `AT` is either a *node ID* or index (when prefixed by `@`), and is optionally followed by the run of nodes. If not specified (and when it makes sense according to the operation's type), `RUN` defaults to 1.

> **Node IDs** are numbers starting from 1, automatically assigned to each node from first to last, in the order they get added to the chain. So, if you start with nodes `ARZDC`, you can predict that `A`'s ID will be 1, `R`'s will be 2, etc. Once a node gets an ID, it never changes; so, node IDs are immutable, just like nodes themselves. **Node indexes** instead refer to the 0-based index of a node in the context of a specific version; so, in `v0` = `ARZDC` character `D` has index=3 (and ID=4); but in `v1` = `ARDC` (deletion of `Z`) the same character has index=2 (while still retaining its ID=4). You are free to choose which system to use, but in most cases you will stick to IDs. Indexes can be useful for nodes which get added in complex nestings of operations, when ID prediction could be less handy.

(3) `OPERATOR` (required): the **operator** defines the operation's type. According to the operator, the other relevant parameters are listed in this table:

type	operator	at	run	to	to-run	value
replace	`=`	Y	Y	-	-	Y
delete	`-`	Y	Y	-	-	-
add before	`+[`	Y	Y	-	-	Y
add after	`+]`	Y	Y	-	-	Y
move before	`>[`	Y	Y	Y	-	-
move after	`>]`	Y	Y	Y	-	-
swap	`<>`	Y	Y	Y	Y	-
annotate	`:`	Y	Y	-	-	-

> Run by default is always 1 except in add where it is 0.

(4) `"VALUE"`: a **text value** for those operations requiring it. For instance, a replacement operation must provide a text to insert for the replaced range defined by `AT` and `RUN`. Alternatively, move operations (move before, move after, swap) have a second pair of coordinates, `TO` and `TORUN`, which work like `AT` and `RUN`. For instance, `1>]3` is a move-after (`>]`) operation, which moves character 1 after character 3.

(5) `^RANK`: a metadatum representing the **probability rank** for the interpretation connected to this operation. A value=0 means unspecified, values above 0 are e.g. 1=highly probable, 2=probable, 3=less probable, etc., using as many levels as you require.

> When not zero, the rank also generates a feature with the same name and a value equal to the rank's value. This allows this metadatum to bubble up into the operation's output, and thus be part of the data attached to each generated version. The rank is thus a *projected feature*, using a single policy. Source features too get projected in a similar way, when present, using a first-single policy.

(6) `[NAME="VALUE"...]`: name=value pairs representing metadata **features**, separated by space. When the feature is flag-like (boolean), you can just use `NAME` without operators and value. Further syntax details:

- the operator separating the name from the value varies according to the set policy of the feature:
  - `=` represents a **multiple** feature, which can occur 0-N times in its set.
  - `:=` represents a **single** feature, which can occur 0-1 times in its set.
  - `==` represents a **first-single** feature, which is treated as single the first time it is added to the set, and then as multiple. This can be used to replace a set of features with another one. For instance, when adding source-related features, the next operation first removes all the features named "source" from the targeted set, and then adds all the new ones.

- **global feature** names must start with an asterisk (e.g. `*version`). A global feature is a feature attached to the chain data context as a whole, rather than to specific nodes in it. Specific nodes are determined by the operation type: for instance, a deletion targets the node being deleted, while an insertion targets the nodes being inserted.
- a feature name prefixed by `!` means that all the features with that name must be **removed**. In this case you just specify the feature name after this prefix.
- a feature name ending with `^` means that that feature will be short-lived, i.e. once added, it will be removed the next time features get updated (i.e. at the next operation). This is useful for instance when adding the `*version^` feature, which should tag a single output as representative of a named text version.
- a value containing **spaces** must be included in double quotes.

Note that features get accumulated as each operation is executed on the chain. So at any stage in the evolution of the text you get all the features accumulated up to it, unless you remove any of them in an operation, according to the set policy used.

Example

Let us see an example. If you recall our sample `ARZDC` snapshot, we could provide the following operations to interpret it (the line prefixed by 👉 represents the version generated by each operation, and the base text):

- 👉 `v0` : `ARZDC`

(1) `3- [*log=\"delete Z\" reason="don't like Z"]` : this is a delete operation targeting node with ID 3, i.e. the third character of our base text (`Z`). Notice that for this demo we have added a couple of mock features: a `log` single feature which has a diagnostic purpose, representing the operation being executed in a human-friendly form; and a `reason` feature, which might represent the reason for that operation. The input of this operation is the base text (`v0`), and the output is a new version, which will automatically be tagged as `v1` (=the numeric suffix is just increased).

- 👉 `v1` : `ARDC`
- 📚 features:
  - context: log=delete Z
  - node #3: reason=don't like Z

(2) `2=V [*log=\"replace R with V\" *version^:=alpha]` : this is a replace operation targeting node with ID 2, i.e. the second character of the base text (`V`), being replaced by `V` . Besides the `log` feature, here we add a `version` single feature, which means that we want to pick the outcome of the operation as a text version proper, named `alpha` . This feature can thus be used to define which of the versions generated by each operation should be promoted to what we envisage as a version of the text, rather than just a reconstructed stage in the process of its evolution. Also note that here we are going to use the version output by the previous operation as the input of this one, which is the default; that's why we do not require to explicitly define neither the input nor the output version tags for it.

- 👉 `v2` : `AVDC`
- 📚 features:
  - context:
    - log=delete Z
    - log=replace R with V
    - version=alpha
  - node #3: reason=don't like Z

(3) `6=B [*log=\"replace V with B\"]` : this is a replace operation targeting node with ID=6, which is the ID of the newly added node `V` in the previous operation (as remarked above, node identifiers are predictable, as they get assigned automatically by incrementing a shared ordinal number).

- 👉 `v3` : `ABDC`
- 📚 features:
  - context:
    - log=delete Z
    - log=replace R with V
    - log=replace V with B
  - node #3: reason=don't like Z

(4) `(v1:) 2=P [*log=\"replace R with P\" *version^:=beta]` : this is yet another replace operation, but this time it does not use the last generated output as its input. Rather, it is based on version `v1` (`ARDC`); so, it's the second branch of replacements stemming from the same base text character `R` .

- 👉 `v4` : `APDC`
- 📚 features:
  - context:
    - log=delete Z
    - log=replace R with V
    - log=replace V with B
    - log=replace R with P
    - version=beta
  - node #3: reason=don't like Z

> Note that when you start from a previous version, the chain is smart enough to generate a unique version number, without colliding with existing ones. The output version number is equal to the input version number + 1, or to the next available number when this already exists.

(5) `4<>5` : this is a swap operation, swapping the position of two characters with ID 4 (`D`) and 5 (`C`). As you can see, swap here best captures the meaning of this operation, and is a higher order operation. A swap is just a special case of movement, which in turn could be represented with a deletion and an insertion; yet, should we go that down, we would lose the immediate meaning represented by a more synthetic and human-friendlier "swap".

- 👉 `v5` : `APCD`
- 📚 features:
  - context:
    - log=delete Z
    - log=replace R with V
    - log=replace V with B
    - log=replace R with P
  - node #3: reason=don't like Z

(6) `(v3:) 4<>5 [*version^:=gamma]` : this is the same swap operation which now affects another branch in the evolution of the text; the operation has as its input `v3` (`ABDC`), so in the end the output will be `ABCD` .

- 👉 `v6` : `ABCD`
- 📚 features:
  - context:
    - log=delete Z
    - log=replace R with V
    - log=replace V with B
    - log=replace R with P
    - version=gamma
  - node #3: reason=don't like Z

So in the end, these operations represent 7 variants of our text:

- `v0` : `ARZDC`
- `v1` : `ARDC`
- `v2` : `AVDC`
- `v3` : `ABDC`
- `v4` : `APDC`
- `v5` : `APCD`
- `v6` : `ABCD`

Additionally, each version has its set of features, as derived by accumulating them from the various operations which generated it.

> 🛠️ Technical note: each chain operation derives from `ChainOperation<T>`, which in turn is derived from `VarOperationMetadata`. All operations get translated into calls to lower-level chain editing methods. The chain operations take care of finding the nodes corresponding to their location arguments. All the operations work on a chain operations **context** (`ChainOperationContext<T>`), which includes the chain itself and a set of related data, like last version tag, context features (for global features), node features, and all the details about each step in processing the text.

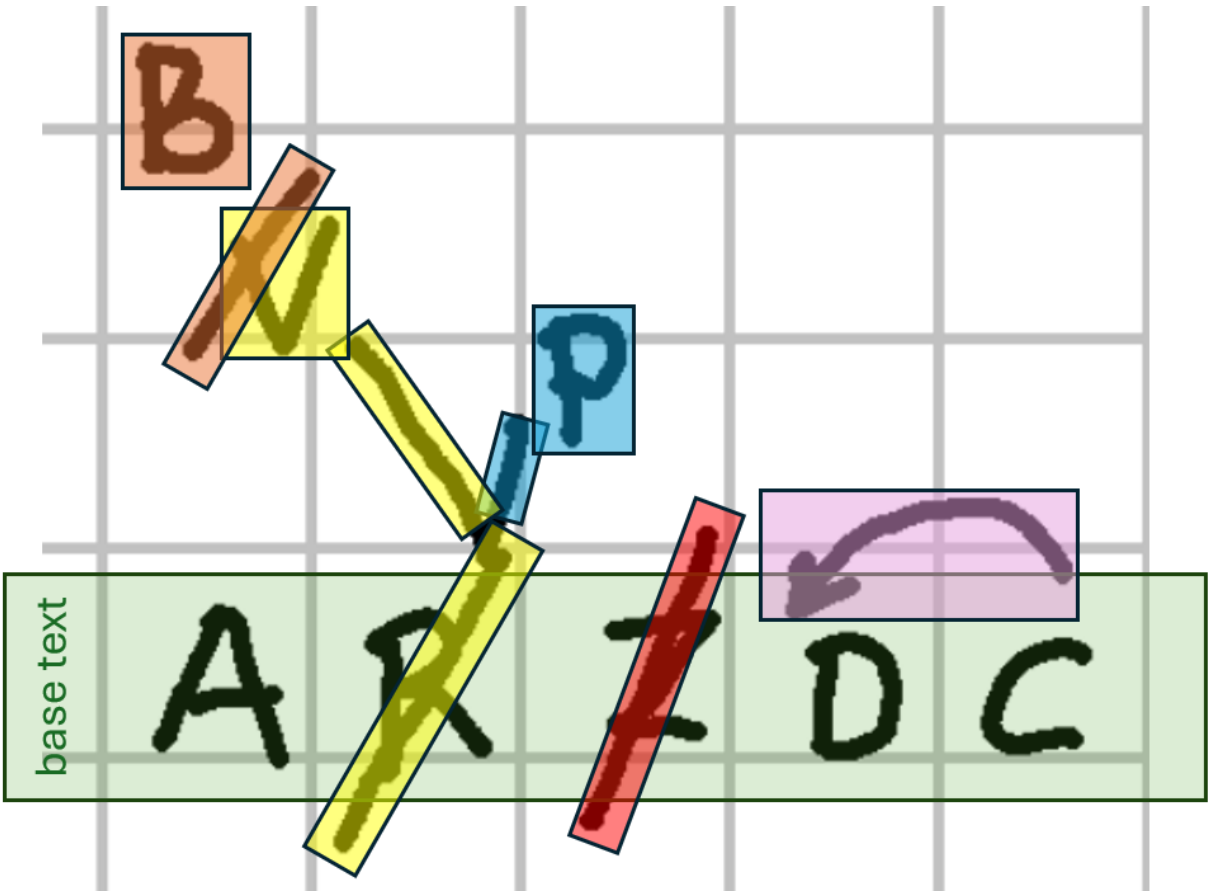
Diplomatic Model

On the diplomatic side, our model should provide an at least approximate graphical representation of the snapshot.

First, note that this is not a requirement for the textual side of the model; we could just implement this without its graphical counterpart, which in some cases might be less relevant or too costly. So this is just an additional part of the model, which is anyway designed to preserve the separation between these two layers.

As we have seen, the core of the textual model is represented by the chain structure, and by operations acting on it. Each operation is an act executed on the text, and in most cases corresponding to some graphical representation in the snapshot.

For instance, consider our usual mock snapshot:



Here I have highlighted each portion of visuals with a different color, corresponding to the operations represented by the snapshot:

- the red bar on top of `Z` represents its deletion.
- the yellow bar on top of `R`, continued to the left up to a `V`, represents a replacement: `R` is replaced by `V`. The visuals here are 3: two segments and a `V` letter.
- the orange bar on top of `V` and the `B` drawn above it are the visuals corresponding to another replacement. This time, `V` is replaced by `B`.
- the magenta arrow on top of `DC` represents a movement operation: `C` is moved before `D`, or alternatively, `D` and `C` are swapped. As in many other cases, here the choice depends on the one which best fits our interpretation.

So, at this level of abstraction, any annotation in our snapshot can be graphically represented by a set of visuals, corresponding to any mixture of geometric shapes and/or text, whatever its complexity.

As annotations are modeled as operations, this means that we can add the graphical representation of each operation as a set of visuals linked to it. This is essentially what is done by the diplomatic portion of the model, which adds these metadata to operations:

- `g` : the SVG `g` element graphically representing the operation this diplomatic information is attached to. SVG being XML, this is just a string in the object model. SVG here is the ideal candidate for this graphical representation, because it's a W3C standard for vector and interactive graphics in HTML pages. As an XML dialect, SVG can represent any geometrical form or text, with all its visual features, just with some text; and this is compatible with both HTML and XML, including of course TEI. Also, it can be made interactive and animated by controlling each of its elements via JavaScript. An SVG `g` element is a group, which may contain any type and number of shapes or texts, or even other groups. So, a single `g` element for each operation can represent any complex visuals for it, as a single unit.
- `features` : a set of generic features attached to the operation as a whole and related to its diplomatic aspects. For instance, we might want to add information about ink, size, etc. Features are modeled as already seen for the textual portion (name/value pairs).
- `element features` : this is a dictionary of features sets, keyed under SVG element IDs. The dictionary allows to attach features to a specific element of the operation visuals, rather than to their whole set. To this end, you assign an arbitrarily chosen ID to every SVG element you want to attach features to, and then add as many features as needed to it under the key of its ID. For instance, we might want to add a specific set of features to only the orange bar among the two visuals building the group representing the replacement of `V` with `B` .

So, this model not only links visuals to each operation, but also attaches optional metadata on the diplomatic side, both to the operation as a whole, or to any parts of its visuals. Yet, the model is very simple, because it rests on just text: SVG code for the visuals, and name=value pairs for features.

Animation

Additionally, this bidimensional, geometric model can be further extended by adding the dimension of *time* in the form of animations.

In depicting our interpretations of a snapshot, tracing back text versions to the actions performed by its author, animations can play a significant semantic role in the context of this model. Of course, they belong to the same realm of interpretation we have already distinguished in that model, but a digital publication can greatly benefit from them.

With reference to our architecture, animations must be designed no longer at the level of each single operation, but rather within the scope of the whole snapshot. For instance, think about a `B` inserted between `A` and `C` : we might want to animate this insertion by moving `B` between these two letters, which in turn would require to be moved in opposite directions, so to make space for the new one. So, this goes beyond the scope of a single operation, and is rather contained in the snapshot as a whole.

Animations and Steps

As we have seen, a chain contains the representation of many versions of what we consider the "same" text, as defined by a set of operations. Each operation generates a new state for the text, representing a **step** towards the "final" state for the text, as reconstructed by our interpretation.

While there is a new step for each operation, usually not all these steps represent what we consider as a true **version** of the text. Anyway, in the model this is just a detail, implemented with a specific metadatum which tags some of the steps as representing a version.

The purpose of animations is visally hinting at the transition between two steps; so, animations are tightly coupled with steps. This has a number of implications, which help shape the model of animations in a simpler tool:

- each animation is *tagged* with the step ID corresponding to its target state. For instance, if an animation must animate the transition between `v3` and `v4`, its tag will be `v4`. While many operations can stem from the same step (for instance, two different variants branching from the same base at different times), each operation always produces a different, uniquely identified step. This is what is used to link animations with steps.
- each animation deals with *all the visuals present in the snapshot up to its reference step, included*. So, if an animation refers to `v4`, and the steps preceding it are `v0`, `v1`, `v2` and `v3`, all the visuals introduced by these steps will be present and available for the animation. As the animation is a transition between the state represented by its source step and that represented by its target step, this in practice means that it is going to provide entrance- (and possibly side-) effects for the visuals in the target step. Thus, in our example we can assume that the visuals up to `v3` are all present and visible in the snapshot, while those of `v4` are present (as injected by executing the operation leading to `v4`), but not yet visible. It is the task of the animation to bring them into visibility with the desired transitions. For instance, a line representing a stroke on a letter to mean its deletion might be progressively revealed like a pen's trait, with a point which stretches into a segment, becoming longer until it reaches its final dimension. In this case, we start with this invisible segment, and apply an animation which gives the live effect of "drawing" it.

> In an autograph, visuals in fact accumulate on the snapshot as they get added to it step by step. Once added, unless we aim for some more abstract visualization their place no longer changes. So, introducing animation here is just a matter of animating the transition between a step with existing visuals to another one with additional visuals. This reflects the physical nature of the model: a snapshot is a set of graphical signs on a material support, and once these signs are traced, there is no way of undoing this action. Of course one can change his mind and graphically represent this by e.g. drawing a stroke on them, or even overwriting them; but once a visual is there, the very fact of its presence cannot be deleted.

- *animations are an optional feature*: the model can work even without them, and still retain its essential representative power. This allows different projects to choose whether to add this additional layer of data, or just be happy with the basic one. A snapshot without animations is just a sequence of still frames; whereas one with animations provides transitions between these stills, by means of value interpolation. Metaphorically, this allows moving from a gallery of single shots to a movie. When properly used, such animations convey more meaning rather than just being a fancy addition: they can help in leading the reader's attention to the crucial stages of a transformation happening right before his eyes, thus helping in understanding the evolution of a text along the time axis by actually using time for this purpose.

GSAP Model

Animations are complex; yet, given that we don't need (nor desire) overtly complex features in our general-purpose model, we can leverage some existing software libraries to define a generic model for them.

Among the freeware libraries targeting web, one of the most popular and well-suited for these requirements is represented by GSAP. The basic concepts and entities used there can be easily imported in our model.

GSAP is based on *tweens*. A **tween** is just a *property setter*, capable of animating any property, like for instance the position of an element, its size, rotation, color, etc. Its name clearly comes from the fact that animation essentially interpolates *between* two different states of an object.

In GSAP, a tween object is created with a function specifying either its start point (`from`, assuming the current state as the end point), or its end point (`to`, assuming the current state as the start point), or both (`fromTo`).

Each of these functions gets these arguments:

1. a `selector` to define the target element(s).
2. a `vars` object, with all the properties/values pairs you want to animate (e.g. `rotation: 360`). This can also include some special properties, whose names are reserved to specify animation behaviors, like `duration`,

`delay`, `ease`, etc.

In turn, tweens are contained in **timelines**, which can be further configured with `vars`, as for tweens. Timelines orchestrate the animations represented by tweens, coordinating their timing and order.

To add a tween to a timeline, the timeline object has corresponding methods (`to`, `from`, `fromTo`). This adds a third optional parameter, position, a string with its own syntax, used to precisely control where things are placed.

Animation Model

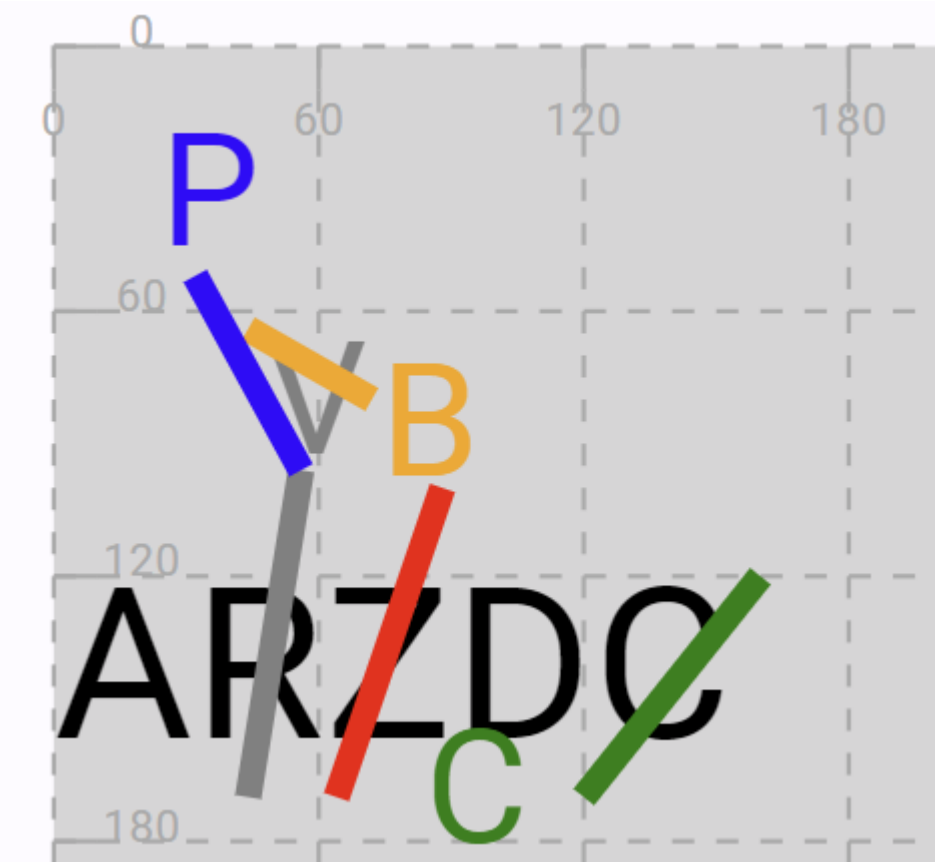
The animation model here can be designed in compliance with a basic usage of GSAP:

- `timelines` (`AnimationTimeline`): a dictionary of 1 or more timelines, keyed under version tags. Each version can thus have its own animation, as it's selecting and reordering a subset of operations on the chain. Each timeline contains a set of tweens.
  - `tweens` (`AnimationTween`):
    - `label`: a human-friendly label for the tween.
    - `note`: a free text note about the tween.
    - `type`: whether it animates from current state to another one (`to`), vice-versa (`from`), or specifying both initial and final states (`fromTo`).
    - `selector`, usually the target element ID. This can be the ID of any of the elements in the SVG code of any operation with a visual representation in the whole snapshot.
    - `vars` (`AnimationVars`, a map where keys are strings and values are strings, numbers, or booleans): a free object with variables to be animated, whose values can be scalar or other objects. As GSAP is JavaScript-based, the object can be encoded as a JSON string. Anyway currently we limit the logic to scalars, as it seems there is no need for more complex values here.
    - `position`: this is just a string.

> A timeline can be easily transformed into a set of instructions for GSAP. Essentially, a tween is created with the timeline's function corresponding to its type (`to`, `from`, `fromTo`), which gets as parameters selector, vars and position from the corresponding tween's properties. In tweens, both label and note have no usage, except facilitating the users work in editing the timeline.

Snapshot Example

In this example I am going to represent the mock snapshot described in the documentation, split into parts for more readability. Its rendered view is shown below:



The underlying storage format for snapshots in this software system is JSON, which has the advantage of being extensible and compact, and very fit to the JavaScript language used in web applications. In fact, JSON is right the way JavaScript represents its code objects as text.

Size and Image

A snapshot first has a size. Optionally, it can have an image (with the same size) with some type of visual representation of its carrier: it might just be a photographic image, or a digital drawing extracted from it, or any other kind of surrogate facsimile representation.

So, the following JSON snippet specifies the snapshot view size (in pixels) and the background image to show as a watermark. The image is displayed behind the snapshot visuals, and you can also specify the opacity level (ranging from 0=transparent to 1=100% opaque) you want for it if you want it to fade out more or less with relation to those visuals.

> In JSON, each object is delimited by braces and can have any number of properties. Each property has a name and a value, which can be a scalar value like a string, a number, or a boolean value, or another object or an array (=list) of items. Arrays are delimited by square brackets. Names are between double quotes, and so are string values. So here there is an object including two other objects under properties named `size` and `image`. In turn, `size` has numeric properties `width` and `height`; and `image` has a string `url` property, and a numeric `opacity` property.

```
{
  "size": {
    "width": 800,
    "height": 600
  },
  "image": {
    "url": "https://via.placeholder.com/800x600",
    "opacity": 0.8
  },
}
```

> In this example, I am using a mock image service to get an image of the desired size (as specified in the image URL) to play with. Of course, in real-world you will use your own image from some other URL.

Text

The base text follows as an array of chain nodes. When you are entering text as a string, the nodes array is automatically built by the editor from it.

Each chain node is an object, having:

- a *numeric ID*. This is automatically assigned by software in a predictable way: for the base text characters, it is just the ordinal number of each character: so the first character is 1, the second 2, and so forth. Then, should other characters be added by insert or replace operations, they will get the next number available, thus ensuring that all the characters have a unique numeric ID in the context of this snapshot, and that this ID reflects our interpretation about the evolution of the text.
- a *label*. This is the display value for the character, in our case equal to the character itself.
- an *index*: this is the zero-based index of the character in the source text.
- *attached data*, which in our case corresponds to a single character.

> In its backend implementation the chain structure is templated for its data type, as we might want to reuse it to chain types different from single characters. So we might have chains of characters, strings, numbers, or even complex objects. Anyway, in this application we just use the maximum granularity level, represented here by the character, as its data item.

```
{
    "text": [
    {
      "id": 1,
      "label": "A",
      "index": 0,
      "data": "A"
    },
    {
      "id": 2,
      "label": "R",
      "index": 1,
      "data": "R"
    },
    {
      "id": 3,
      "label": "Z",
      "index": 2,
      "data": "Z"
    },
    {
      "id": 4,
      "label": "D",
      "index": 3,
      "data": "D"
    },
    {
      "id": 5,
      "label": "C",
      "index": 4,
      "data": "C"
    }
  ],
}
```

Optionally you can specify the base text's CSS style (any valid CSS value string) and options (of type `SvgBaseTextOptions`):

```
{
"textStyle": "font-size: 48px",
  "textOptions": {
    "offset": {
      "x": 0,
      "y": 120
    },
    "lineHeightOffset": 8,
    "charSpacing": 0,
    "spcWidthOffset": 0
  },
}
```

Here I set a big font size, as the snapshot contains a few letters; and I offset the base text top-left corner by 0,120 to leave some space for annotations on top of it.

In general, the **base text options** can include these properties:

- `lineHeightOffset` * (`number`): offset to add to each line height.
- `charSpacing` * (`number`): space to add between characters.
- `spcWidthOffset` * (`number`): offset to add to the space width.
- `offset` (`Point`): offset for the top-left corner of the base text wrapper rectangle.
- `minLineHeights` (`Record<number, number>`): minimum line heights for each line number (1-N). You can specify a minimum line height for each line number to ensure that its height is at least the specified value. This is useful for making room between lines for annotations that may exceed the base text.

Operations

Operations are objects inside an `operations` array property. They are listed in their order of application.

Here I am going to list them one at a time, preceded by its essential text-based representation (which covers only a subset of its properties, but is useful for quick data entry or diagnostic purposes).

Delete Z

```
3- [*log="delete Z" reason:="don't like Z"]
```

This operation deletes the character node with ID=3 (`Z`), and injects two context-wide features: a multiple global one (`*log`) and a single node-scoped one (`reason`).

This is because in this example we want to output a sort of per-operation log to keep track of operations execution, and to attach to the deleted `Z` node the reason for its deletion.

The *log* feature is global and multiple (`setPolicy=0`), which means that whenever it gets added it attaches itself to the result as a whole, rather than to specific node(s); and that there can be many features with the same name. This allows accumulating log features for each operation, thus keeping track of their execution. Here this has only a diagnostic purpose.

The *reason* feature is not global and thus node-scoped; it it the operation type which determines the nodes it will attach to, corresponding to the affected nodes. Here, the affected node is the `Z` node being deleted. Also, this feature is single (`setPolicy=1`), which means that whenever we add a new feature with the same name, the old one

will be removed.

Finally, the diplomatic (visual) representation has a `g` property with the SVG code for the visuals of the operation. In this case, these are just a diagonal red stroke on the `z` character.


```
{
  "features": [
  {
    "isGlobal": true,
    "name": "log",
    "value": "delete Z",
    "setPolicy": 0
  },
  {
    "name": "reason",
    "value": "don't like Z",
    "setPolicy": 1
  }
  ],
  "id": "c705f7c65a",
  "type": 1,
  "at": 3,
  "run": 1,
  "diplomatics": {
  "g": "<g id=\"del-z\"><line x1=\"64\" y1=\"170\" x2=\"88\" y2=\"100\" stroke=\"red\" stroke-width=\"6\"/></g>"
  }
},
```


Replace R with V

```
2=V [*log="replace R with V" *version^:=alpha]
```

This replaces `R` with `V`, injecting the usual *log* feature, and a *version* feature. The version feature is meant to promote the operation's output to what we consider as a true version of the text. Every operation produces a new step, but not all the steps correspond to versions; rather, they accumulate until one is reached.

For this reason, version is global (attached to the snapshot result as a whole), single (it replaces an existing version feature), and short-lived. The last property means that the feature will be automatically removed as soon as the next operation is executed. This is the desired behavior here, as we tag as "version" a text state, and as soon as we move away from it that's no longer that version.

The operation visuals deserve more comment (see below).

```
{
  "features": [
  {
    "isGlobal": true,
    "name": "log",
    "value": "replace R with V",
    "setPolicy": 0
  },
  {
    "isGlobal": true,
    "isShortLived": true,
    "name": "version",
    "value": "alpha",
    "setPolicy": 1
  },
  {
    "name": "x",
    "value": "48",
    "setPolicy": 1
  },
  {
    "name": "y",
    "value": "65",
    "setPolicy": 1
  },
  {
    "name": "style",
    "value": "font-size:36px;fill:gray",
    "setPolicy": 1
  }
  ],
  "sources": [],
  "id": "66aed192fc",
  "type": 0,
  "at": 2,
  "run": 1,
  "value": "V",
  "diplomatics": {
  "g": "<g id=\"rep-RV\"><line x1=\"44\" y1=\"170\" x2=\"56\" y2=\"96\" stroke=\"gray\" stroke-width=\"6\"/></g>"
  }
},
```


The operation visuals here are distributed in two different layers of the snapshot: one is the base text, because we are adding a new node, `V`; another is the diplomatic layer, where we add any visuals representing the operation.

The new node we add should be outside the automatic layout of the base text. This just displays each character after another in a line; but here we want to represent the added character just like it appears in the snapshot, above the text and at the edge of a stroke on the replaced character.

We do not want to represent this `v` character as a mere visual on the diplomatic layer, because it is part of the text: it's a node in it. Yet we have to remove this node from the automatic layout and place it where it belongs. This will allow it to be treated as all the other text nodes in the snapshot.

To this end, we add some node-scoped features to the operation, which will be thus injected in `v` : *x*, *y*, and *style*. A node having *x* and *y* is placed at these coordinates, thus outside the automatic layout. Also, the style specifies its color (gray) and its size (a bit smaller than the base text).

💡 Features names *style* and *class* are reserved to represent the SVG element attributes `style` and `class` respectively.

On the diplomatic layer instead we just have a grey stroke on `R` .

Replace V with B

```
6=B [*log="replace V with B" reason:="like B"]
```

This is again a replace operation, so it essentially looks like the previous one: a new node `B` and an orange stroke on `v` :

```
{
    "features": [
    {
    "isGlobal": true,
    "name": "log",
    "value": "replace V with B",
    "setPolicy": 0
  },
  {
    "name": "reason",
    "value": "like B",
    "setPolicy": 1
  },
  {
    "name": "x",
    "value": "74",
    "setPolicy": 1
  },
  {
    "name": "y",
    "value": "70",
    "setPolicy": 1
  },
  {
    "name": "style",
    "value": "font-size:36px;fill:orange",
    "setPolicy": 1
  }
  ],
  "sources": [],
  "id": "e5610844bd",
  "type": 0,
  "at": 6,
  "run": 1,
  "value": "B",
  "diplomatics": {
  "g": "<g id=\"rep-VB\"><line x1=\"44\" y1=\"64\" x2=\"72\" y2=\"80\" stroke=\"orange\" stroke-width=\"6\"/></g>"
  }
},
```


Replace R with P

```
(v1:) 2=P [*log="replace R with P" *version^:=beta]
```

This is yet another replacement, so similar to what we have above. This time the color is blue and we also mark the result as a version ("beta").

Also notice that the input state for this operation is not the last generated step, as usual, but rather "v1". This is because we have a branching from the original base text R: the first branch replaces it with V and then B; the second branch replaces it with P. So, we could not start from the previous step, which is the last operation on the first branch; these branch are mutually exclusive, as they represent alternatives.


```
{
  "features": [
  {
    "isGlobal": true,
    "name": "log",
    "value": "replace R with P",
    "setPolicy": 0
  },
  {
    "isGlobal": true,
    "isShortLived": true,
    "name": "version",
    "value": "beta",
    "setPolicy": 1
  },
  {
    "name": "x",
    "value": "24",
    "setPolicy": 1
  },
  {
    "name": "y",
    "value": "18",
    "setPolicy": 1
  },
  {
    "name": "style",
    "value": "font-size:36px;fill:blue",
    "setPolicy": 1
  }
  ],
  "sources": [],
  "id": "503c1e3a94",
  "type": 0,
  "inputTag": "v1",
  "at": 2,
  "run": 1,
  "value": "P",
  "diplomatics": {
  "g": "<g id=\"rep-RP\"><line x1=\"56\" y1=\"96\" x2=\"32\" y2=\"52\" stroke=\"blue\" stroke-width=\"6\"/></g>"
  }
},
```

Swap D and C

```
(v3:) 4<>5 [*version^:=gamma]
```

Finally, we swap `D` and `C` in the branch generated with step "v3". This generates version "gamma". This time the visuals represent a green arc and the C letter in its new position.

Note that this letter is not a node of the base text, because the snapshot view stays fixed to the base text nodes, "ARZDC", while still adding new nodes as required. This reflects the material source, where essentially nothing which is drawn on the support can be undone; you may well overwrite it in any way, but once written, it is there. The operations are rather represented as annotations referring to it.

So, the snapshot view already has the C node in its original position. The newly written C is just a sort of mirrored image of it, a pure visual artifact which together with the arc represents its movement.


```
{
  "features": [
    {
      "isGlobal": true,
      "name": "version",
      "value": "gamma",
      "setPolicy": 1
    }
  ],
  "sources": [],
  "id": "ea8ffab3d4",
  "type": 6,
  "inputTag": "v3",
  "at": 4,
  "run": 1,
  "to": 5,
  "toRun": 1,
  "diplomatics": {
    "g": "<g id=\"swap\"><path style=\"fill:none;stroke:green;stroke-width:3\" d=\"M 140,160 A 24,8 0 0 2 130,166 24,8 0 0
  }
}
```


Recap

In the end, our operations are:

1. `3- [*log="delete Z" reason:="don't like Z"]` : visuals = red stroke on `Z` .
2. `2=V [*log="replace R with V" *version^:=alpha]` : visuals = grey stroke on `R` . The newly introduced text node is grey, smaller, and located just above the top of the stroke.
3. `6=B [*log="replace V with B" reason:="like B"]` : visuals = yellow stroke on `V` . The newly introduced text node is yellow, smaller, and located to the right of the stroke.
4. `(v1:) 2=P [*log="replace R with P" *version^:=beta]` : visuals = blue segment connected to the grey stroke on `R` . The newly introduced text node is blue, smaller, and located at the top of the segment.
5. `(v3:) 4<>5 [*version^:=gamma]` : visuals = green stroke on `C` and a smaller green copy of it below `Z` and `D` .

The visuals usually reflect the snapshot's visual appearance in an approximate way; they might be drawn on top of an image of the carrier, or even be directly derived from it by vectorization.

The degree of approximation can greatly vary according to the purpose of the digital representation, from a highly similar vector-based representation to a purely symbolic representation of the underlying operations. On the other end, we might even omit visuals altogether and just limit ourselves to the text layer.

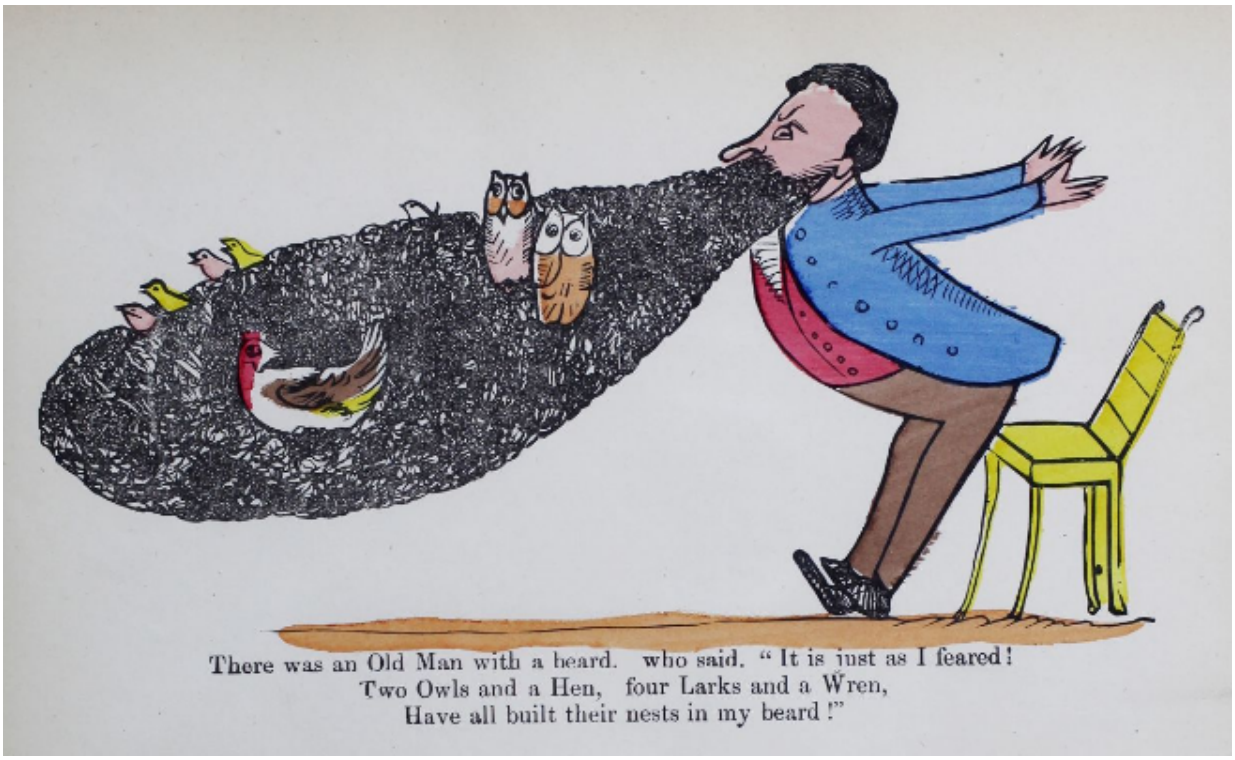
Each visual provided with an ID, i.e. all the base text characters and all the visuals from operations metadata whose element has an `id` attribute, are also interactive objects on the page, so you could have your presentation software interact with the user via mouse events like hovering or clicking on them.

In the end, the snapshot view provides users with an overview of the text creation and transformation processes, as derived from blending two layers, which are separately modeled:

- an *objective* data layer, the digital representation of the snapshot as it can be described from its carrier. This is the set of all the visuals as they appear on it. Of course, even at this level there might be a level of subjectivity (for instance in reading a single sign), but this usually is a corner case, and anyway it can be encoded using features (like a probability rank) about it.
- a *subjective* data layer one, the philological interpretation of the snapshot, which is described by operations. These guide us in reading the snapshot, by interpreting the operations represented by its annotations so that they are filtered and ordered to get all the text versions we deduce from it. Additionally, we describe this recipe for getting our text versions from the ingredients on the table of the carrier in such a way that we effectively get operational instructions for the underlying chain data structure. This makes the system capable of generating all the text versions with all their metadata, up to the granularity level of a single character.

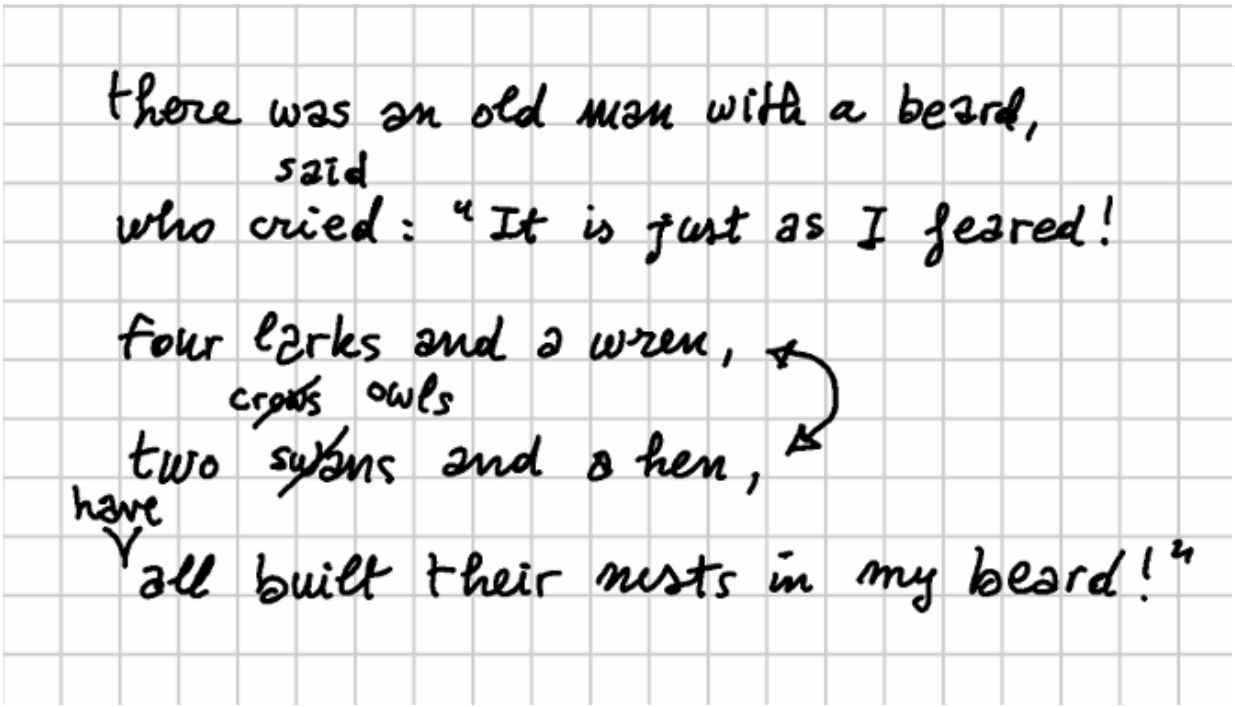
True Text Example

Let us make an example with a true text. Consider the first limerick in The Book of Nonsense by Edward Lear (p.7 of the 1875 edition you can find at http://purl.flvc.org/fsu/fd/FSULearBook_012):




```
There was an Old Man with a beard,
who said, "It is just as I feared!
Two Owls and a Hen,
four Larks and a Wren,
Have all built their nests in my beard!"
```

Let's pretend that we had an autograph like this:



This is our **snapshot**. We now use the snapshot editor to enter its data in the GVE text model.

Base Text

Let us start by defining the **base text** (`v0`). Our mock snapshot is very simple: by looking at it, we can clearly see that this corresponds to the text which was first laid out following the printed sheet lines. So, we can define the base text as:

```
there was an old man with a beard,
who cried: "It is just as I feared!
four larks and a wren,
two swans and a hen,
all built their nests in my beard!"
```

> 👉 Note that the base text essentially is just a string, so that newlines are represented in it just like any character node, corresponding to a single newline character (LF). In the UI this is visualized as an arrow-down character.

In the GVE UI, we can just paste this text to get the identifiers assigned to each node in the base text. In the screenshot below, I have selected the nodes corresponding to the word "cried", ranging from 40 to 44 (a `40x5` span coordinate, as displayed at the top of the screenshot):

> 💡 To select a range, click on the first node, and then Ctrl+click on the last one.

Operations

We can now start interpreting the snapshot by inferring editing **operations** from it.

Of course, operations are the product of our subjective interpretation, while compatible with the objective data provided by the snapshot. Here, we assume that after changing a couple of words, the author inserted "have" for metrical reasons, and this produced what we regard as a staged version, named "alpha".

We then assume that later, the author read this text again, and he decided to swap verses 3-4 to get a better progression (both numeric and in syllabic mass). Finally, he also changed "crows" into "owls". The final outcome is what we regard as staged version "beta".

So, more formally, we have:

1. replace "cried" with "said";
2. replace "swans" with "crows";
3. insert "have" before "all" (for metrical reasons);
4. swap verses 3-4;
5. replace "crows" with "owls".

Let us formalize this interpretation, representing it with the DSL syntax used for essential GVE operations:

1. `40x5="said"` : replace "cried" with "said" (`v1`). Note that this introduces new nodes in the graph, one for each character of the word "said". We can predict the identifiers assigned to these new letters because the algorithm is designed to make it possible: we just get the maximum ID number, and add 1 to it to get the next ID. So here we can predict that `s` =151, `a` =152, `i` =153, `d` =154. Alternatively, we can just use the UI to execute operations up to the desired point, and then look at the resulting output, which allows you to get the IDs of each node while also inspecting all their metadata.
2. `99x5="crows"` : replace "swans" with "crows" (`v2`). Again, we can predict the new IDs (155-159).
3. `116+["have " [*version^="alpha" reason="metrical"]` : insert "have" before "all" (`v3`): staged version alpha. As we are inserting a separate word, we also add a space after it. Also, note that here we add a couple of operation metadata:
   - `version` : this assigns a staged version metadatum to the output of this operation, i.e. we mean that after this operation we got a version of the text we consider as a real "Fassung" in the history of our text. Of course, we might have thousands reasons for doing this, one of the most obvious being a previous printed edition of this book among our witnesses. Note that by definition whenever we promote an output to a staged version this implies that this refers to the resulting text as a whole, and that it will be made void as

soon as we change again that text. This is why in our syntax we add the `*` prefix (=global metadatum) and the `^` suffix (=short-lived).

- `reason` : the reason we supposed for this operation. The set policy here (as specified by `=`) is multiple, which is the default. We might have several reasons for an operation; and in this case, each would be a different metadatum with same name (`reason`) and different values.

4. `72x23<>95x21` : swap verses 3-4 (`v4`). Note that the swap just takes as reference node the first one of each verse, because due to the nature of range coordinates we just specify a start point (the AT) and a span count (the RUN). So, whatever the identifiers of the nodes in each verse, all verse nodes will be selected. For instance, here "four larks and a wren," (23 characters including LF) include the original sequence of nodes with IDs ranging from 72 to 94; while "two crows and a hen," (21 characters including LF) contains IDs 95-98 (for "two "), 155-159 ("crows"), 104-115 (" and a hen,").
5. `155x5="owls" [*version^="beta"]` : replace "crows" with "owls" (`v5`): staged version beta.

> For diagnostic purposes, all the operations also get a global `log` feature which gets accumulated for the text as each operation is executed.

So, here we are describing all changes step after step. Each operation output produces a new version, but only two of them are marked as staged, with names `alpha` and `beta` respectively. In short:

```
40x5="said" [*log="said for cried" x="72" y="140" style="font-size:20px;fill:red"]
99x5="crows" [*log="crows for swans" x="120" y="210" style="font-size:20px;fill:red"]
116+["have " [*log="insert have" *version^="alpha" reason="metrical" x="60" y="270" style="font-size:20px;fill:red"]
72x23<>95x20  [*log="swap verses"]
155x5="owls" [*log="owls for crows" *version^="beta" x="210" y="210" style="font-size:20px;fill:red"]
```

The output versions are:

- `v1` ("said" replaced "cried"):

```
there was an old man with a beard,
who said: "It is just as I feared!
four larks and a wren,
two swans and a hen,
all built their nests in my beard!"
```

- `v2` ("crows" replaced "swans"):

```
there was an old man with a beard,
who said: "It is just as I feared!
four larks and a wren,
two crows and a hen,
all built their nests in my beard!"
```

- `v3` ("have" added before "all"; staged as `alpha`):

```
there was an old man with a beard,
who said: "It is just as I feared!
four larks and a wren,
two crows and a hen,
have all built their nests in my beard!"
```

- `v4` (swapped verses 3-4):

```
there was an old man with a beard,
who said: "It is just as I feared!
two crows and a hen,
four larks and a wren,
have all built their nests in my beard!"
```

- `v5` ("owls" replaced "crows"; staged as `beta`):

```
there was an old man with a beard,
who said: "It is just as I feared!
two owls and a hen,
four larks and a wren,
have all built their nests in my beard!"
```

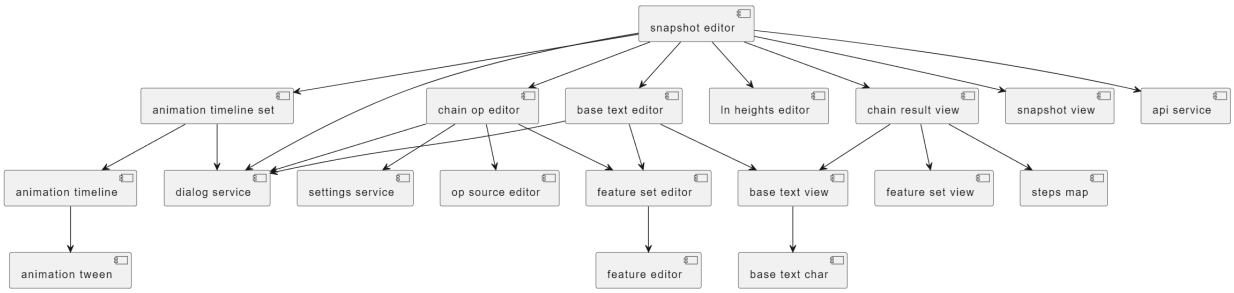
This is the minimal data we need to represent our text versions. We can then go on, and add visuals and even timelines; anyway, until now all the snapshot data could be entered via text: a base text and 5 lines representing a batch of edits is all what is required to feed the chain engine and let it generate all the versions, each with its own metadata attached to either specific text nodes, or to the version as a whole.

GVE Core Library

The `gve-core` Angular library contains core frontend components to be used by GVE client apps. As already remarked, the frontend UI for editing and visualizing snapshots is highly modular, and is built by aggregating many components together. These components are organized into software libraries.

The GVE core frontend software library contains all the components used for editing a snapshot in the context of an Angular-based web application, which in the case of GVE is represented by a Cadmus-based application.

The diagram below represents the main components in this library and their relationships:



The top-level component here is the snapshot editor. This provides a complete UI for editing all the data contained by a snapshot, and usually is all what you need to integrate a full snapshot editor in an Angular application.

In turn, the snapshot editor is composed by a lot of other components, covering all the snapshot features.

- editing snapshots
- editing operations
- editing features

Snapshot - Editing

The snapshot editor is used to fully edit a snapshot, and as such it's the topmost UI component in its library. All the other components in the library are either directly or indirectly used by it. Essentially, it allows you to edit all the parts of a `Snapshot` object.

In general, to create a snapshot you follow these steps:

1. examine and interpret your source, so you can come up with a text to start with (the base text), and any number of operations changing it into all the versions you define in your interpretation of their annotations.
2. type or paste the multiline text to be used as the base text.
3. optionally, provide an image to use as the snapshot background. In this case, you would typically add visuals which overlap it.
4. optionally, if needed add features to one or more characters of the base text.
5. describe each single operation on the base text leading to a change in the text.
6. optionally, describe the visuals of each operation in its corresponding diplomatic sections in the editor. You might do this from scratch, or import an SVG drawing from some third party SVG editor.
7. optionally, define the animations for some or all of the versions.

In defining operations, often changes on text as deduced from the carrier can be interpreted in a **linear** way, so that you apply operation 1 to the base text, leading to version 1; then, apply another operation 2 to the output (version 1), leading to version 2; and so forth, like here:

```
v0 -> v1 -> v2 -> v3 ...
```

Anyway, it might also happen that your interpretation ends with **branching**, so that at a certain point the text output by an operation might be the input of two different branches, which change it in alternative ways. In this case, you will just have to change the default input version tag for your branching operation: rather than just continuing from the previous operation in a linear way, you will start from any of the versions generated previously:

```
v0 -> v1 -> v2 -> v3 ...
            v2 -> v4 ...
```

In this example, the operation generating v4 does not get its input from `v3` (which would be the default), but rather from `v2`, thus creating a new branch. So, there is no limit to the complexity of branching, as far as it's just represented by a different input tag for the corresponding operation starting a new branch.

Also notice that, while technically each change in (what we regard as a unique) text produces a new version of it (i.e. simply a different output), not all the versions are eligible to represent what we define as a **staged version** of the text in a more philological meaning. Rather, the various changes produced by each operation get accumulated like steps towards one of these staged versions.

For purely practical reasons, in the editor we just use the term "version" for the output of each operation, which will be automatically tagged as `v1`, `v2`, etc. Yet, you can add the specific `version` feature (with your own tag) to the operation whose output is to be regarded as a specific staged version of the text. So for instance the version "v3" (which is just the output of its operation) might be regarded as the staged version "alpha". So, in your interpretation you are just setting the path towards staged versions by defining sequences of one or more operations changing the text.

The UI is articulated in 4 tabs:

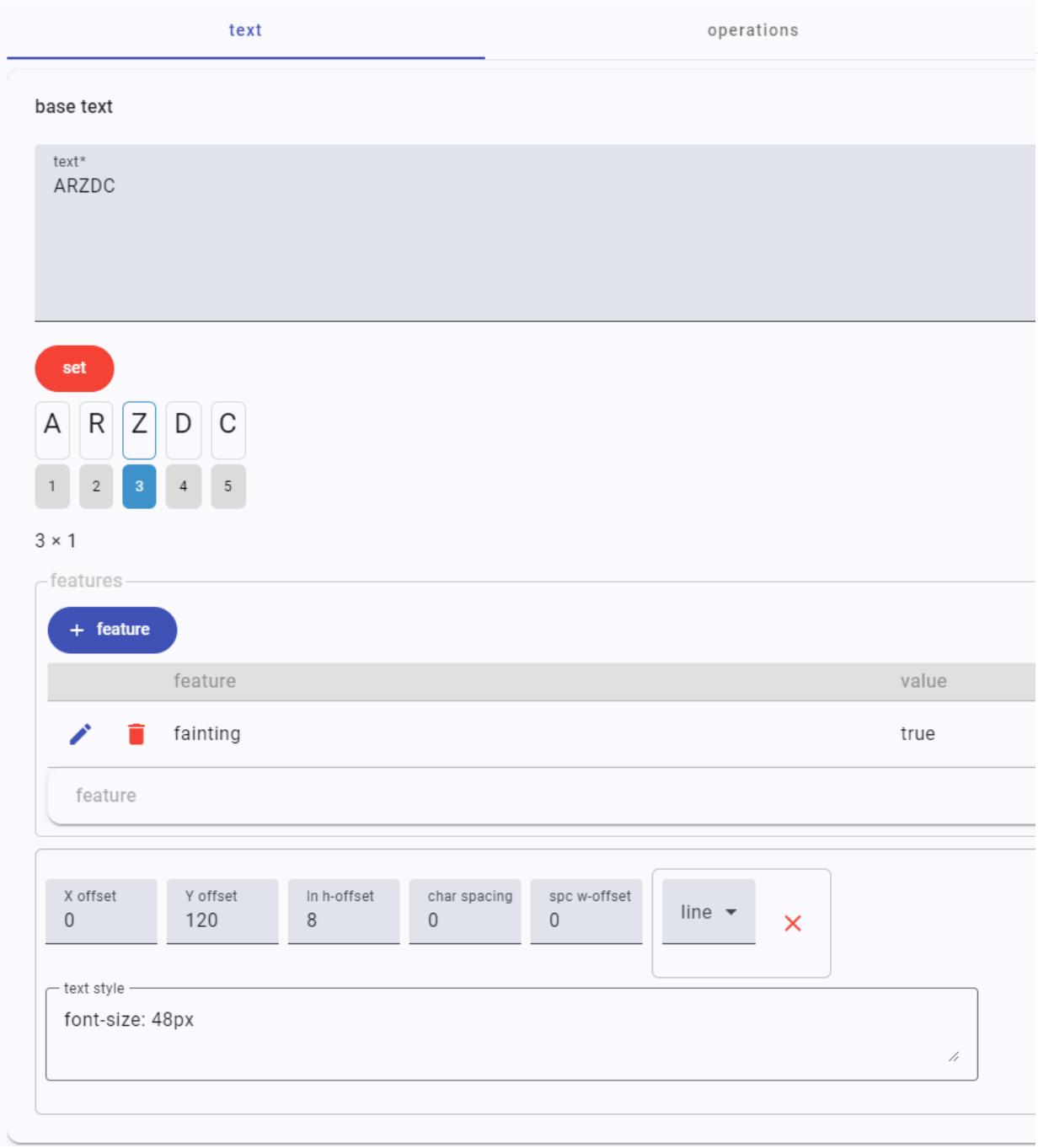
- Snapshot - Editing
  - Text

- Operations
- Image
- Animation
  - Special Properties
  - Easing

Text

The text tab contains the **base text**. You typically set this once, and then add the other data of the snapshot. Remember that the snapshot provides annotations which can be interpreted as operations changing the base text, which is just what happens to be the starting point of our digital representation. So, "base text" here has no genetic or philological implication; it is just a practical device in our model, i.e. our starting point, even though often it corresponds to some evidence in the carrier (e.g. a text laid on top of the same line, with interlinear annotations).

> ⚠️ It is not advisable to change the base text once you have started adding metadata on top of it, even if a patch function can be later added to the editor to help you in such cases. If you change the base text, all the annotations will be removed, as they make sense only when referred to a specific base text. Anyway, changing a base text later is a rare event if occurring at all, as setting the base text is the first, foundational act for defining a snapshot, and all what comes later is just based on this first state of the text.



To set the text, just type or paste it, and click the set button. Once you have done it, the text will be displayed character by character in an interactive diagram, where you can click each single character to get its ordinal-based coordinate, or add features to it.

> As the text is split into its characters, please ensure you remove any unwanted artifacts from it, like sequences of more than a single space, etc. Also, do not add empty lines or use spaces to represent visual aspects, as these belong to another level of our representation and we should not misuse characters for the sake of encoding some specific appearance.

In the characters list, you can also click to **select** one character, and then control+click to select the last character of a range; in this case, the coordinates of the selected range will be shown. This is useful when you want to quickly get the coordinates of the text targeted by operations.

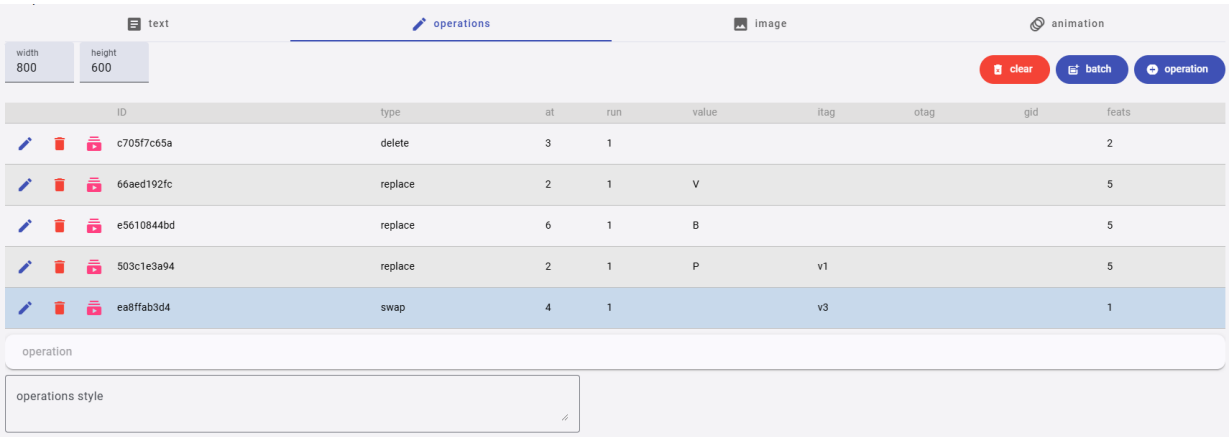
The text tab also allows to change some **general metadata** for the text:

- an open set of features.

- a set of visualization metadata for its preview:
  - `X-offset` : the optional offset to add to the X coordinate of the text origin.
  - `Y-offset` : the optional offset to add to the Y coordinate of the text origin.
  - `ln H-offset` : the optional offset to add to each calculated line height. You can set this if you want to get more default vertical space between the base text lines.
  - `char spacing` : the optional offset to add to each calculated character width. You can set this if you want to get more default horizontal space between each character of the base text.
  - `spc W-offset` : the optional offset to add to each calculated space width. The width of a space character is automatically calculated according to the default size of the base text; should you want to customize it, you can set this offset accordingly.
  - `min. line heights` : the optional minimum line height of any specific line in the base text. When you just want a constant line height, leave this option out. When instead you want to set the minimum height of one or more specific lines, pick the line number to change, and enter its height. You can use the red `x` button to remove all the custom heights at once, or set to 0 a line height to remove its customization from the set.
  - `text style` : the CSS code representing the *inline* style to be applied to the text root `g` element. The content of this field will be output in the `style` attribute of that `g` element.

Operations

The operations tab contains the most relevant data of a snapshot, i.e. the operations which change its text, starting from the base text and then following one or more paths to generate all the versions for it.



The tab contains a list of operations, in their execution order. For each operation, it shows:

- the operation ID. This is a unique code generated automatically.
- the operation type.
- the coordinates of the target nodes: `at` =first node, `run` =count of nodes starting from `at` .
- the text value associated to the operation, if any. For instance, an insert operation will have here the value to be inserted.
- the tags of the versions to be used as the operation input (`itag`) and output (`otag`). In real world scenarios, you will occasionally have to change the input tag when you are branching variants, but you will never change the output tag.
- the group ID (`gid`) of the operation. This is for logically grouping operations into a semantically unique set, but has no practical effect on the chain.
- the count of features (`feats`) associated to the operation.

Using the **buttons** next to each operation, you can:

- edit the operation.

- delete the operation.
- run the operations on the base text up to the current operation to see the result in the preview panel at the bottom of the snapshot editor.

Below the operations list a **style** text box allows you to enter the CSS code for the inline style to assign to the operations root `g` element. This is rarely used, but can be handy when you want to set some style shared by all the operations at once.

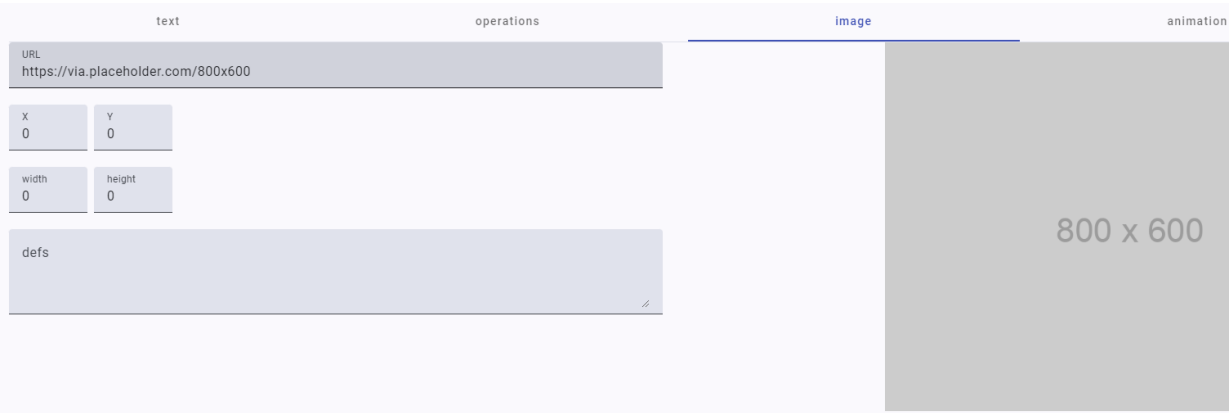
In some cases, when starting with a new snapshot you might want to add operations in **batch**. This can be much quicker than editing each operation at hand, even if it does not allow to fully define all the possible metadata of an operation (like its visuals). Anyway, this is a handy way to quickly fill the operations list starting from a simple text, either you type or paste it.

To do this, just type or paste a text having one line per operation, following the specific syntax designed for them, which is summarized on the right, and hit the `batch add` button. A `clear` button also allows you to remove *all* the operations in the snapshot, should you want to quickly re-add them in a batch. This way, you can enter your text representing the batch, generate the operations from it, and quickly fix any errors and repeat until you are satisfied with the result.

Once you have added the operations in a batch, you can edit each of them to add or refine its data.

Image

The image tab allows you to optionally define an image to be displayed behind your snapshot visuals. Typically this might be a properly sized image of the snapshot's carrier, like a handwritten sheet of paper, so you can see it as a watermark reference behind the operations visuals.

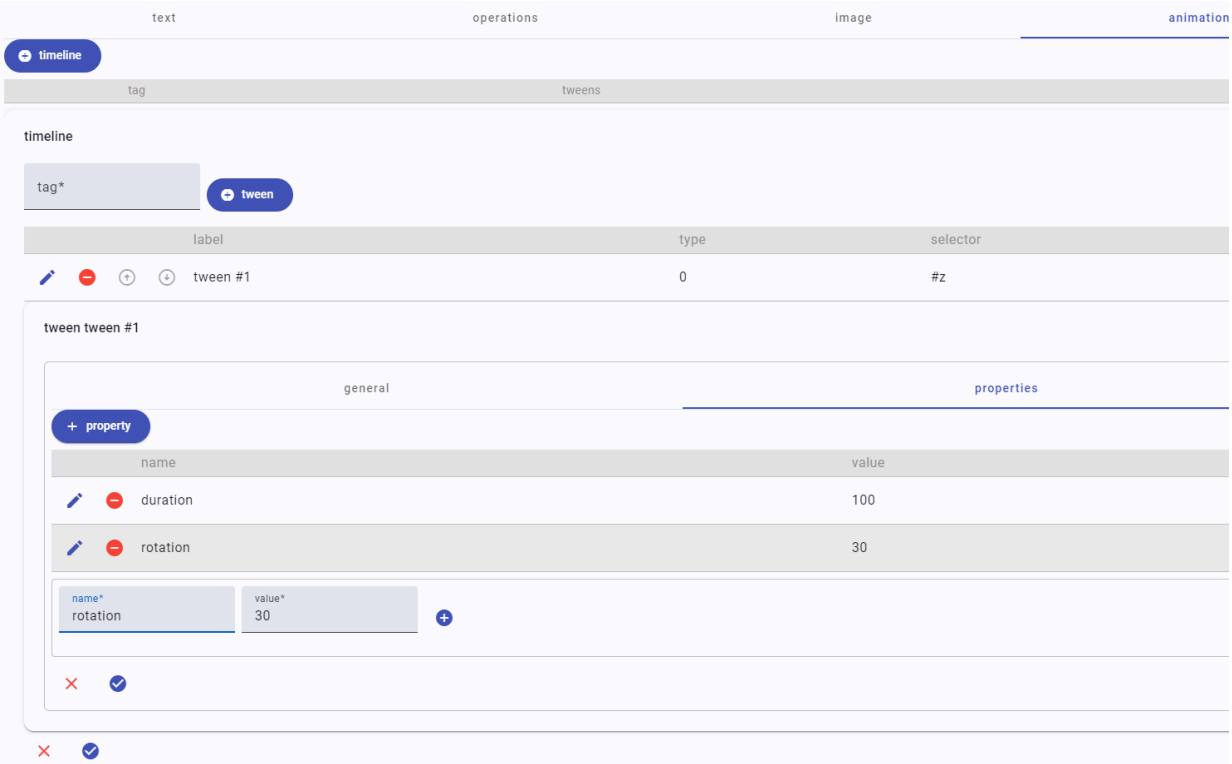


Available data are:

- `URL` : the image URL. A preview of the image will be shown on the right (unless of course the URL is not accessible).
- the optional canvas to fit the image into, defined by `X` , `Y` , `width` and `height` . If not specified (=everything is 0), the image will have the same size of the SVG container, and placed at its top-left corner (0,0).
- the optional SVG `defs` code to be used in the view. This usually is an advanced SVG feature you might use to define reusable SVG geometries for your operation visuals, like e.g. an arrow which is drawn many times in the same way. In most cases you can leave this blank.

Animation

The animation tab contains the optional animation timeline(s) you might want to add to your snapshot versions. You can define a timeline for each generated text version, using the version ID as its tag. Each timeline includes one or more tweens. The model here follows GSAP documentation.



Timelines are used to create easily adjustable, resilient sequences of animations. When you add tweens to a timeline, by default they'll play *one after another*, in the *order they were added*. Timelines share most of the same special properties that tweens have like `repeat` and `delay` which allow you to control the entire sequence of animations as a whole.

A **tween** has:

- `label` : a human-friendly label.
- `type` : one of `to` , `from` , `fromTo` . In most cases you will be using `to` , which starts at the element's current state, and animates to the values defined in the tween; `from` is like a backwards `to` , animating from the tween values back to the element's current state; `fromTo` instead defines both the start and end values.
- `position` : an optional position. The position parameter is the secret to building sequences with precise timing. There are a variety of position parameters that we can use to position tweens pretty much anywhere. The most commonly used position parameters are:
  - absolute time (seconds) from the start of the timeline: `tl.to(".class", {x: 100}, 3);` .
  - gap: `tl.to(".class", {x: 100}, "+=1");` (seconds); `tl.to(".class", {x: 100}, "+=50%");` (percent of total duration).
  - overlap: `tl.to(".class", {x: 100}, "-=1");` (seconds); `tl.to(".class", {x: 100}, "-=25%");` (percent of total duration).
- `selector` : the selector for the tween's target. Typically, this is an element ID, but any valid CSS selector can be used.
- `note` : an optional note about this tween. You might want to use this to describe your tween effect using words.
- `properties` : the properties changed by the tween, or special GSAP properties controlling the animation. Almost any property can be animated: transforms, colors, padding, border radius, etc. Just remember to camel-case the properties - e.g. `background-color` becomes `backgroundColor` .

> 💡 Even if you can animate virtually everything, choose your target properties wisely. When possible use transforms and opacity for animation rather than layout properties like "top", "left" or "margin". Transforms can be used to move your elements around, scale them up and spin them around; transforms and opacity are also very performant, because they don't affect layout.

In GSAP there are shortcuts for transforms:

- `transform: translateX(N)` = `x:N` or `xPercent`
- `transform: translateY(N)` = `y:N` or `yPercent`
- `transform: scale(N)` = `scale:N` ; same for `scaleX` , `scaleY` .
- `transform: rotate(N)` = `rotation:N`
- `transform: skew(N)` = `skew:N`
- `transform-origin: POS N` = `transformOrigin: "POS N"`

Further, some commonly used properties are:

- `opacity`
- `autoAlpha` (shortcut for opacity and visibility)
- `duration`
- `repeat` (-1 for infinite)
- `delay`
- `yoyo` (boolean)

Special Properties

The most common special properties are:

- `duration` : duration of animation (seconds; default 0.5).
- `delay` : amount of delay before the animation should begin (seconds). You can also use `repeatDelay` to add a delay to the start of any repeat iterations.
- `repeat` : how many times the animation should repeat (-1=infinite repeat). Repeat is often paired with `yoyo` in order to reverse the direction each cycle.
- `yoyo` : if `true` , every other repeat the tween will run in the opposite direction (like a yoyo).
- `stagger` : time (in seconds) between the start of each target's animation (if multiple targets are provided). To get more control, wrap things in a configuration object which can have more properties in addition to most of the special properties that tweens have.
- `ease` : controls the rate of change during the animation, like the motion's "personality" or feel. Default: `power1.out` .
- `onComplete` : a function that runs when the animation completes.

Easing

Under the hood, the "ease" (special property `ease`) is a mathematical calculation that controls the rate of change during a tween. Use the ease visualizer to pick your own.

For most eases you'll be able to specify a type. There are three types of ease: `in` , `out` and `inOut` . These control the momentum over the course of the ease. In practice, they mark the slower part, e.g.:

- `power1.in` : start slow and end faster, like a heavy object falling.
- `power1.out` : start fast and end slower, like a rolling ball slowly coming to a stop.
- `power1.inOut` : start slow and end slow, like a car accelerating and decelerating.

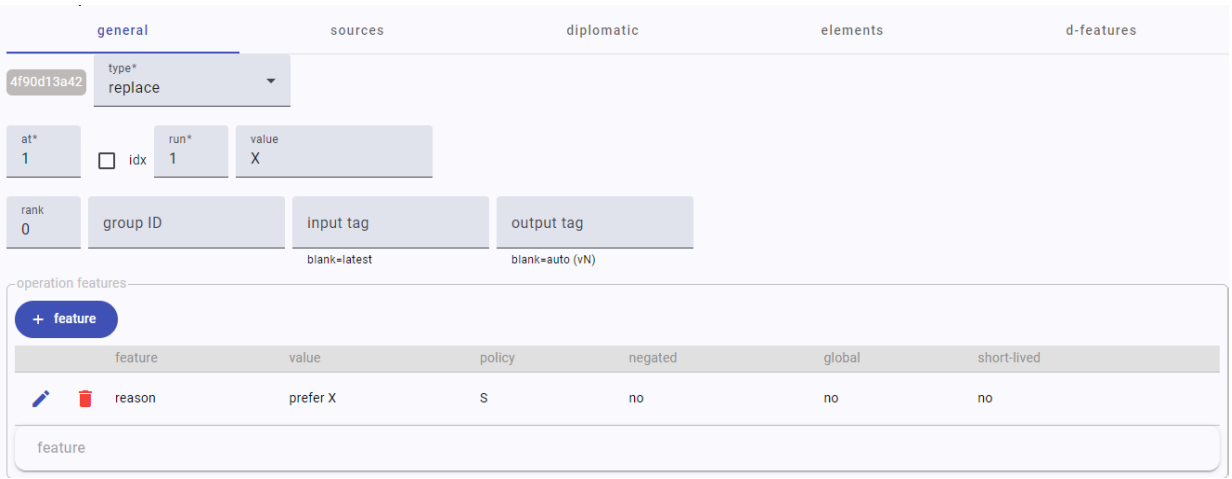
Operation - Editing

The operation editor is used to edit a single snapshot operation. Its UI is divided in 5 tabs:

- Operation - Editing
  - General
  - Sources
  - Diplomatic
  - Elements
  - D-Features

General

The general tab contains general metadata about the operation. An operation changes the received text, or only affects its metadata in the case of an annotate type.



- a 10-characters ID, automatically generated. This is not editable.
- the operation `type` : replace, delete, add-before, add-after, move-before, move-after, swap, or annotate. According to the type you select here the UI might slightly change (for instance, move and swap require an additional parameter).
- `at` : the coordinate of the first character targeted by the operation. In the base text, each character receives an ordinal number used as its ID (1=first, 2=second, 3=third...). Any character node added later will get the next available ordinal. You can refer to the base text view to get the coordinate. In some cases you might prefer to express this as a (zero-based) index, rather than using an ID. In this case, check the `idx` option next to it.

> The ordinal nature of the character IDs ensure that they represent the original ordering of the base text characters, which get a continuous sequence of numbers from 1 to N; and that any character node added later (by insertion or replacement) gets an ID which is granted to be greater than N. This also allows a predictable numbering scheme. Anyway, the order of the characters in the sequence is defined only by their links, and each version (in the technical sense of output) has its own links set.

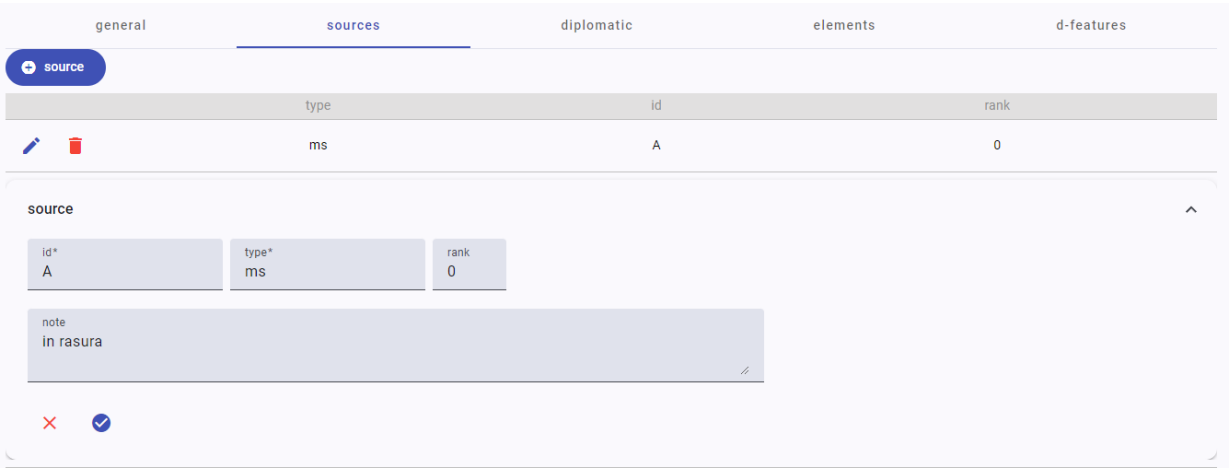
- `run` : the count of characters targeted by the operation (1-N) starting from `at` . So, the minimum run value is 1. Usually, it refers to the extent of a word, but it can address sequences of any length.
- `rank` : a numeric value representing the certainty rank for this operation: 0=not specified, 1=most probable, 2=less probable, etc. Usually this stays at 0, unless you have to express a level of uncertainty in the interpretation represented by the operation.
- `group ID` : the ID of a group this operation belongs to. This is just a tag used to group operations into a higher conceptual unit, should this be required. In this case, just use an arbitrarily defined value for the group ID.

> Even though this is not a requirement, whenever dealing with arbitrary IDs it is recommended that you follow strict naming conventions. For instance, limit the characters to `a` - `z`, digits, underscore, and dashes, and avoid very long identifiers.

- `input tag` : the version tag for the input of this operation. Usually this is left blank, meaning that the last generated version in the operations sequence will be used as the input. If you need to branch the versions starting from a previous operation in the sequence, specify its tag here.
- `output tag` : the version tag for the output of this operation. In most cases this is left blank, which generates an automatic tag for the operation output with form `vN`, where `N` is an ordinal number. In practice, just leave this always blank.
- `features` : the features of this operation. Here you can add as many features as you want. Operation features have a richer model, including not only name, value, and set policy, but also the following properties:
  - `negated` : a negated feature means that any existing feature with the same name should be removed. In other terms, this is used to delete an existing feature from the target set if present.
  - `global` : a boolean value indicating whether this feature globally refers to the operation context as a whole, rather than to its affected nodes. When a feature isn't global, it will be assigned to the nodes targeted by its operation, rather than to the whole text features set. As operations are executed, they accumulate on nodes and/or the whole text, unless their set policy dictates a different behavior. In the end, this will produce a rich set of highly granular metadata, mapped either to specific portions of the text or to the text as a whole.
  - `short-lived` : a boolean value indicating whether this instance is short-lived. When true, this means that the feature will be removed on the next update, i.e. when the next operation in the sequence is executed.

Sources

The sources tab contains data about the sources of this operation.



A source might be a document, a person, or any other entity regarded as its source. For instance, if Schiller makes an annotation represented by an operation, the source of this operation will be Schiller. An operation can have zero or more sources.

Each source has:

- `id` : an arbitrary string used to identify the source.
- `type` : a string defining the source type (e.g. document, person, etc.).
- `rank` : a numeric value representing the certainty rank for the identification of this source, used like the operation's rank (0=not specified, 1=most probable, 2=less probable, etc).
- `note` : a free text note.

Diplomatic

The diplomatic tab refers to the visuals connected to the operation. For instance, a stroke on a letter might be the visual representation corresponding to a delete operation.

Operation visuals can be defined by any geometrical shape and/or text on top of the carrier surface. Technically, they are represented by a single SVG `g` (=group) element, which can include any number and type of SVG descendant elements. So here you can enter any valid SVG code, provided that it has a single `g` element as its unique root element.

You can add an `id` attribute to any of these SVG elements, if you want to target them for animations or for linking features to them.

Also, by convention you can add a `_t` suffix to any `id` whenever you want to add animations targeting the visual. When you add this suffix, the corresponding visual opacity is programmatically set to 0 (=transparent) when the software builds the SVG code for the snapshot viewer. This is because the visual is meant to be made visible by some animation, which implies the requirement to start as transparent.

> Using this approach avoids the risk of an unwanted effect due to timing issues in animating objects: should you start with a visible visual, and then make it invisible to animate its entrance, this might produce a so-called FOUC effect, where the object appears for an instant before being made invisible. So, whenever you want to animate the entrance of a visual, just ensure you append the "transparent" `_t` suffix to it and in your animation assume that it will start with its `opacity` =0.

A toolbar at the top of the SVG code editor contains these commands:

- `save` : save the SVG code into a file.
- `load` : load SVG code from a file.
- `copy` : copy the SVG code into the clipboard.
- `set` : set the SVG code from the clipboard.
- `open editor` : open a designated external graphical editor for SVG. The default editor is https://editor.method.ac/, but this can be changed and is a configuration parameter. In the case of this editor, you can copy the existing SVG code if any, paste it into the external editor, draw as desired, and copy back the resulting code. Then, use the set button to set the SVG code from the clipboard.

Elements

The elements tab is dependent from the diplomatic tab, and lists all the SVG elements having an `id` attribute, so that you can attach to them any number of features. This way, you can get maximum metadata granularity also on the visuals side.

- to add or edit features to an element, click its pen button.
- to remove all the features from an element, click the delete button.

You are free to decide for the level of abstraction attached to your visual representation. Visuals can be a highly surrogate and schematic representation, or try to be as faithful as possible to the source image. In the latter case, you might even want to show that image as the background of the geometric representation of visuals.

Note that in most cases visuals should be representing non-textual entities, like strokes, shapes, and drawings of any sort. When the operation visuals include text, because this is being inserted or replaced, you do not need to add text elements to represent it.

Visuals representing texts, i.e. characters, are handled at the text layer, and become part of the output text. So, they will be part of the text passed to the snapshot viewer, just like any other character from the base text. Anyway, you can displace the newly added characters so that they appear in the desired location. For instance, say your autograph text shows a letter X above a letter Y, meant to replace it. In this case, this newly introduced character node, X, will take the place of Y unless you displace it above.

To displace the character(s) introduced by an operation, you can add features to the operation named `x` and `y` with its desired coordinates. These coordinates will be used by the snapshot viewer to anchor the newly introduced text at the starting point they define.

> Similarly, reserved feature names besides `x` and `y` are also `class` and `style`, which add the corresponding attributes to the target element in the snapshot viewer.

D-Features

The diplomatic features tab contains a set of features assigned to the diplomatic description of the snapshot as a whole. So, any diplomatic feature not linked to any specific visual element (e.g. a sheet width or height) should go here.

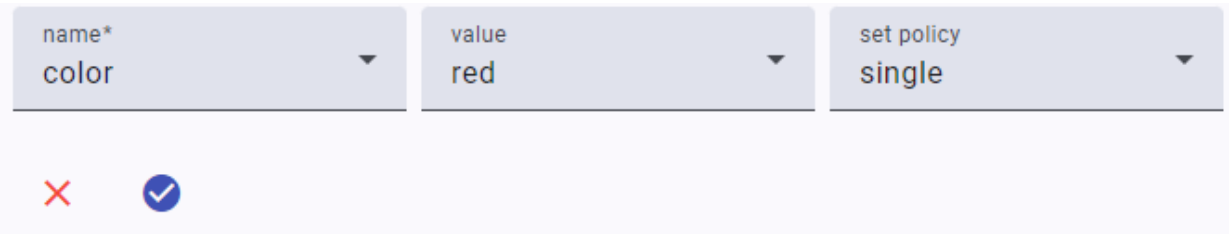
Features - Editing

Features are generic metadata essentially modeled as name=value pairs. Sets of these features are edited via a features set editor, which in turn uses a single feature editor to edit each feature in the set.

Single Feature Editor

The single feature editor is used in the features set editor. This UI allows editing a single feature in the following ways:

- by freely typing name and value.
- by selecting name from a list, while still freely typing value, if you want to limit the features to a predefined set.
- by selecting name from a list, and then value from a corresponding list, for those predefined features having a predefined set of values.



The feature editor contains 3 controls:

- `name` : the feature name.
- `value` : the feature value.
- `set policy` : the features set policy for this feature. This can be `single` , `multiple` , or `single first` (see the features model for more).

By default, you can just type whatever you want as a feature name and value. Anyway, if you rather prefer a closed taxonomy, the client code can customize the editor by passing preset names and values:

- when passing preset names, the name control will turn into a selection control. You will just have to pick an item from the list.
- preset values can be set for all the preset names, or for just a subset of them. This allows having some of the preset names with a free value, and others with a preset value you can pick from a list. In this example, the `color` feature has preset values, while the `size` feature has not.

To **test** the behavior of the component, you have a couple of checkboxes at the top of its test page:

- feature names: when checked, it reconfigures the component to use preset names.
- feature values: when checked, it reconfigures the component to use preset values (for `color` only).

Once you have finished editing, you can click the check button to save your changes. The test page will then show the received JSON code representing the edited feature.

Features Set Editor

The features set editor is used to edit a set of features; in turn, it uses the single feature editor to edit each feature in its set.

The editor shows the full list of its features in a table where each row is a feature. You can click the edit button to edit the feature, or the trash button to delete it. To add a new feature, just click the add feature button.

> Remember that if you add to the set a feature with single/single first policy and another feature with the same name is in the set, that feature will be replaced by the new one.

Optionally, you can use the filter at the top of the table (when present) to quickly filter the rows to match only those whose name includes what you type in it. The filter visibility is controlled by the filter treshold parameter: when it is 0, the filter is always visible; when -1, it is always invisible; otherwise, it gets visible when the number of features is greater than the specified threshold (the default being 5).

To **test** the behavior of the component, you have a couple of checkboxes at the top of its test page:

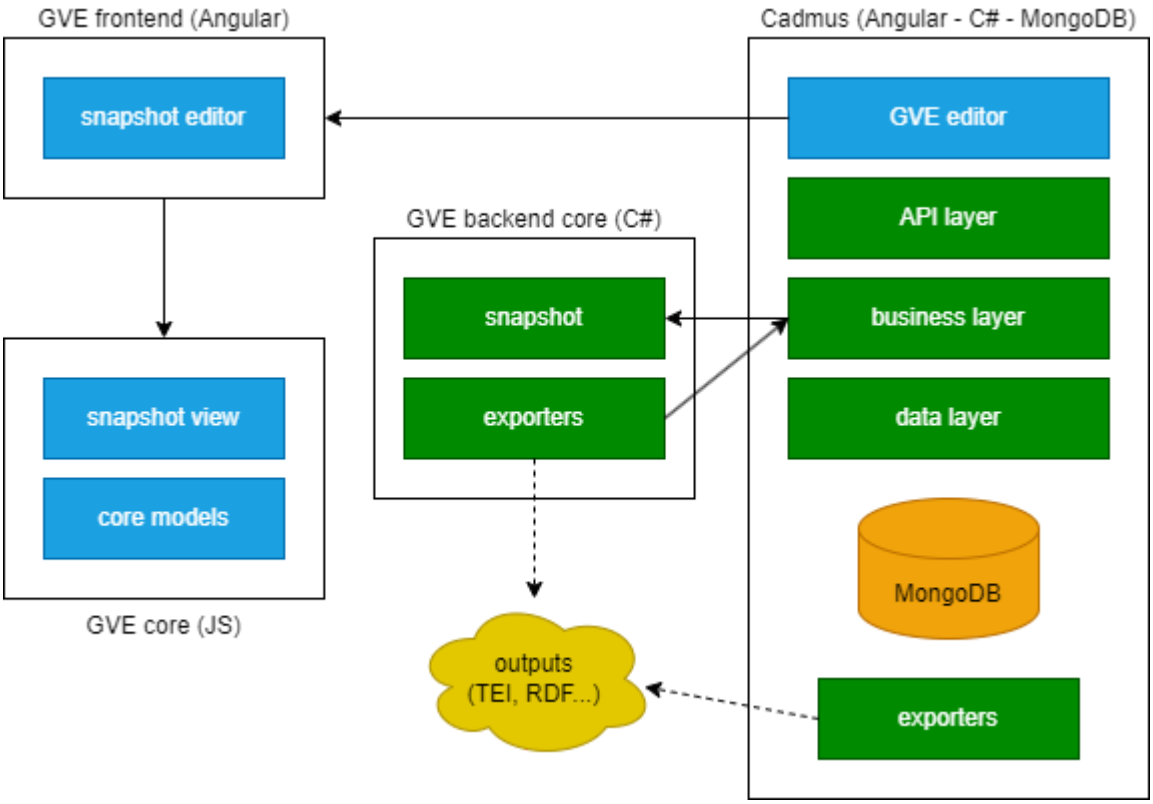
- feature names: when checked, it reconfigures the component to use preset names.
- feature values: when checked, it reconfigures the component to use preset values (for `color` only).
- filter treshold: the value for the filer treshold.

Whenever you change the set, the test page will show the received JSON code representing it.

Architecture

The GVE architecture is based on three main groups of components:

- backend components, which provide the snapshot model and the logic for dealing with it. These are all implemented with C#.
- frontend components, which provide a set of layers to be used both in editing and in displaying snapshots. Core models are in a pure Typescript library; the snapshot viewer is a pure web component, so that it can be easily placed in a web page or integrated in any JS-based UI; the snapshot editor is a set of Angular components providing a full-fledged editor for snapshots.
- Cadmus components, which provide integration with the Cadmus editor. Cadmus is a web-based, modular editor for highly structured data, based on a layered architecture and storing its data in a NoSql, JSON-based database (MongoDB). All its backend layers are implemented in C# language, using ASP.NET for web API endpoints, including CORS, JWT, rate limiting, and auditing. Its frontend is based on Angular. For GVE, this generic Cadmus solution uses GVE-specific models from the corresponding backend and frontend libraries.



GVE Backend

The GVE backend essentially consists of the snapshot model and its core chain data structure. The only language used for backend is C#.

The technical documentation about these components is temporarily located at http://www.fusisoft.it/docs/gve/index.html. The full reference can be found there, while the conceptual documentation has been included and refactored in this site.

Backend Core

The core GVE components are hosted in a single C# library, Gve Text, which has no dependencies on any project-specific component.

All the other backend components essentially are infrastructure.

Demo Infrastructure

These components provide infrastructure for the GVE shell frontend, used to develop and test the frontend snapshot editor components, and the GVE chain demo.

- GVE shell API: API endpoints for the GVE shell frontend. This frontend is a web UI used to develop and test GVE frontend components used in the snapshot editor. The backend API provide chain transformation functionality.
- GVE demo app: a Blazor web application used for playing with the chain data structure. Using Blazor allows to directly access the C# core without an intermediate API layer.

Cadmus Infrastructure

These components provide infrastructure for using the GVE model from within Cadmus. This allows reusing this modular web-based editor to enter data for the whole edition, including but not limited to the snapshots.

- Cadmus part for GVE snapshot: a Cadmus part representing a GVE snapshot. Essentially it is just a wrapper for the snapshot in a form usable by the Cadmus editor.
- Cadmus part seeders for GVE snapshot: a mock data seeder for the Cadmus part for GVE snapshot. Usually, every part in Cadmus has a corresponding seeder, which is used to generate mock data to be used in the editor.
- Cadmus GVE services: services providing the GVE models to the Cadmus infrastructure.
- Cadmus GVE API: API endpoints for the Cadmus GVE editor.

GVE Frontend

The frontend portion provides UI for displaying and editing snapshots.

The UI has been designed into a set of **layers and modules**, with the primary purpose of providing a reusable tool for this and other projects. Currently, there are 4 layers, each corresponding to a different implementation technology:

1. **snapshot core** (TS library, packaged with NPM: `gve-snapshot-core`): this is a pure Typescript library, framework-agnostic, containing just the definitions of the parts of a snapshot model. These definitions are provided in a separate library so that everyone can reuse them even without importing any piece of logic, which is rather provided by upper layers.
2. **snapshot view** (Web Component, packaged with NPM: `gve-snapshot-view`): this is a framework-agnostic, pure web component which can be directly integrated in any HTML page or application to visualize a snapshot. Integrating this component is as simple as typing the name of its tag in the HTML page, and passing it a JS object with data of the snapshot to visualize. Starting from this object, the view takes care of all the complex details relative to generating an interactive SVG-based visualization of the snapshot. As such, this component can be used both in an editing environment, where it works as a preview (and that's how we use it in our editor), and in a presentation environment for end users. This is why it has been implemented with a highly neutral web-based technology with no dependencies from a specific web framework like Angular, React, or the like: this is a pure W3C standard web component.
3. **snapshot editor** (Angular 18+ library, packaged with NPM: gve-core): this is the core component used to fully edit the snapshot model. This is implemented as an Angular library, so that it can be easily integrated into any Angular based web application. In turn, this editor uses the snapshot view to preview the content being edited.
4. **VEdition editor** (Angular 18+ Cadmus-based editor, distributed in containerized Docker images: `cadmus-gve-app`): this is the editor planned for the full content of VEdition, which is not limited to snapshots, but extends to

a whole tree of entities based on it. In this case we adopt Cadmus for its modularity and extensibility, which allows us to build a full web editor on top of an already mature tool, backed by a full-fledged database. In turn, this editor uses the snapshot editor library for editing snapshots, and more libraries for the other entities involved.

This layered design allows to reuse the logic included in each component without having to introduce unwanted dependencies, according to the purposes and environment of reuse.